

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SCUOLA DI INGEGNERIA E ARCHITETTURA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA GESTIONALE

# BEYOND MINIMAX: ADVANCED TREE SEARCH ALGORITHMS AND MACHINE LEARNING FOR SEQUENTIAL DECISION-MAKING — FROM GAMES TO SUPPLY CHAIN OPTIMIZATION

**Tesi di Laurea in Intelligenza Artificiale**

**Relatore:**
Prof. Andrea Borghesi

**Presentata da:**
Giacomo Sacchet

**Correlatore:**
Prof. Lilian Buzer

---

Sessione ottobre 2026
Anno Accademico 2025/2026

# Contents


**I Introduction and Theoretical Background** **4**

**1 Introduction** **4**
- 1.1 Motivation and Context . . . 4
- 1.2 Research Questions . . . 4
- 1.3 Thesis Structure and Contribution . . . 5
- 1.4 Code and Data Availability . . . 5

**2 Theoretical Background** **6**
- 2.1 The Sequential Decision-Making Framework . . . 6
- 2.2 Adversarial Search: Game Trees and Minimax . . . 6
- 2.3 Pruning and Approximation: Alpha-Beta and Depth-Limited Search . . . 7
- 2.4 Sampling-Based Search: Monte Carlo Tree Search . . . 7
- 2.5 Persistent Search and Value Refinement: Descent Minimax . . . 8
- 2.6 Static and Dynamic Evaluation Functions . . . 8
- 2.7 Which Assumptions Do These Algorithms Actually Need? . . . 9

**II Search Algorithms, Neural Evaluation, and Comparison** **9**

**3 Minimax and the Tic-Tac-Toe Implementation** **9**
- 3.1 Project Overview and Rules . . . 9
- 3.2 Initial Architecture . . . 10
- 3.3 Evolution of the AI Player . . . 10
  - 3.3.1 A Random Baseline . . . 10
  - 3.3.2 First Recursive Iteration . . . 11
  - 3.3.3 Final Refactor: Tuple Return Values . . . 11
- 3.4 The Minimax Algorithm: Logic and Implementation . . . 11
- 3.5 Depth Optimization in Minimax . . . 12
- 3.6 Conclusions . . . 13

**4 Connect 4: Depth-Limited Minimax and the 3-Player Extension** **13**
- 4.1 Introduction . . . 13
- 4.2 From Tic-Tac-Toe to Connect 4: Core Architecture . . . 13
- 4.3 AI Level 1: Random Baseline . . . 13
- 4.4 AI Level 2: Heuristics and Informed Scores . . . 13
- 4.5 AI Level 3: The Minimax Algorithm . . . 14
  - 4.5.1 Theory and Complexity . . . 14
  - 4.5.2 Limited-Depth Minimax and Grid Evaluation . . . 14
- 4.6 Optimization and Performance Testing . . . 15
  - 4.6.1 Empirical Analysis of Calculation Times . . . 15
  - 4.6.2 The “One-Second Limit” . . . 15
- 4.7 Game Extension: 3-Player Mode . . . 16
  - 4.7.1 An Alternative Victory Condition . . . 16
  - 4.7.2 Systemic Modifications and Current AI Strategies . . . 16
- 4.8 Final Structure and Conclusions . . . 16

**5 Training a CNN for Sport Image Classification** **16**
5.1 Overview: Two Parallel Tracks . . . 16
5.2 The Dataset . . . 17
5.3 TinyCNN Sport: Iterative Refinement in PyTorch . . . 17
5.3.1 Baseline: Adam at LR = $10^{-3}$ . . . 17
5.3.2 Learning Rate Sweep . . . 17
5.3.3 SGD with Momentum . . . 18
5.3.4 Batch Normalization . . . 18
5.3.5 Learning Rate Scheduler . . . 18
5.3.6 Data Augmentation . . . 18
5.3.7 Better CNN: A Double Convolutional Architecture . . . 19
5.4 Transfer Learning with MobileNetV2 . . . 20
5.4.1 Preprocessing and Architecture . . . 20
5.4.2 Phase 1: Fast Feature Extraction . . . 20
5.4.3 Phase 2: Feature Extraction with Augmentation . . . 20
5.4.4 Phase 3: Fine-Tuning . . . 20
5.4.5 Cross-Phase Comparison . . . 21
5.5 Discussion: Two Frameworks, One Set of Lessons . . . 22

**6 From Image Classification to Game Evaluation: Transitioning to Othello** **22**
6.1 From Classifying Images to Evaluating Boards . . . 22
6.2 Why Othello? The WTHOR Dataset Advantage . . . 22
6.2.1 Limitations of Connect 4 for Supervised Learning . . . 22
6.2.2 The WTHOR Format . . . 22
6.2.3 Othello's Structural Properties . . . 23
6.3 Code Architecture: A Five-Block Layout . . . 23
6.4 Preserving Continuity Across Phases . . . 24

**7 Search Algorithms in the Othello Engine: Minimax, Alpha-Beta Pruning and MCTS** **24**
7.1 Overview . . . 24
7.2 Random Baseline (Strategy 1) . . . 24
7.3 Heuristic Greedy (Strategy 2) . . . 25
7.4 Minimax Search (Strategy 3) . . . 25
7.5 Minimax with Alpha-Beta Pruning (Strategy 4) . . . 25
7.6 Monte Carlo Tree Search (Strategy 5) . . . 26

**8 The Descent Algorithm: Cohen-Solal's Contribution** **27**
8.1 Minimax Strikes Back . . . 27
8.2 Descent: Extending the Search to Terminal States . . . 28
8.3 Implementation: Descent Minimax in the Othello Engine . . . 28
8.4 A Note on Chronology . . . 30

**9 Recap: Comparative Overview of the Six Implemented Strategies** **30**

**10 Integrating the CNN Evaluator: OthelloCNN and Neural-Guided Search** **31**
10.1 Design Rationale: Why Not TinyCNN Sport? . . . 32
10.2 Architecture: OthelloCNN . . . 32
10.3 Board Encoding . . . 33
10.4 Dataset and Augmentation . . . 33
10.5 Training Protocol . . . 33
10.6 Training Results . . . 34

10.7 CNN Integration into Search Algorithms . . . 36
10.7.1 Design Principle: Drop-In Replacement . . . 36
10.7.2 EvaluateGridCNN: Implementation and Sign Convention . . . 37
10.7.3 Minimax and Alpha-Beta + CNN (Strategies 3c, 4c) . . . 37
10.7.4 MCTS + CNN (Strategy 5c): Replacing the Rollout . . . 37
10.7.5 Descent Minimax + CNN (Strategy 6c) . . . 38

**11 Test Methodology** **38**
11.1 Goal and Design Principles . . . 38
11.2 Per-Phase Protocol . . . 39
11.3 Match-up Design . . . 40
11.4 Post-hoc Analysis of Colour Effects . . . 41

**12 Test Results and Analysis** **41**
12.1 Full Results . . . 41
12.2 Sensitivity to the Treatment of Draws . . . 43
12.3 The Algorithm Hierarchy . . . 43
12.4 The Effect of the CNN Evaluator . . . 43
12.5 Computational Cost . . . 44
12.6 Colour Effects Within Phases . . . 44
12.7 Summary . . . 46

**III Conclusions: The Beer Game and the Management Engineering Bridge 46**

**13 The Beer Game as a Markov Decision Process** **47**
13.1 Motivation and Scope . . . 47
13.2 The Beer Game: Problem Description . . . 47
13.3 Formalization as a Markov Decision Process . . . 48
13.4 Structural Differences from the Board-Game Setting . . . 50

**14 Algorithm Transfer Analysis and a Proposed Architecture** **51**
14.1 Algorithm-by-Algorithm Transfer Analysis . . . 51
14.2 A Proposed Architecture . . . 54
14.3 Expected Challenges and Open Questions . . . 55

**15 General Conclusions** **56**
15.1 RQ1: Yes – a Static Evaluator Helps Regardless of Search Paradigm . . . 56
15.2 RQ2: Partial – Some of the Methodology Transfers, Some Does Not . . . 56
15.3 RQ-side: A Qualified Yes – Descent Outperforms Fixed-Depth Search, but Does Not Benefit More from the CNN . . . 57
15.4 Limitations . . . 57
15.5 Closing Remark . . . 58

# Part I
# Introduction and Theoretical Background

## 1 Introduction

### 1.1 Motivation and Context

This thesis develops a search-and-evaluate methodology for sequential decision-making on three board games of increasing complexity, and then asks whether that methodology extends to a problem with no board and no opponent: a supply-chain inventory system, formalized as a Markov Decision Process.

Board games look like a narrow, recreational choice of testbed, but for this purpose they are close to an ideal laboratory. A game state is fully specified; the actions available from it are finite and known; the consequence of an action is exactly determined, or, at worst, drawn from a known distribution; and a completed game resolves unambiguously into a win, a loss, or a draw. None of this holds for the operational problems that motivate including this topic in a management engineering curriculum: an ordering policy, a production schedule, an inventory strategy are also sequential decisions made under uncertainty, but the state is only partly observed, the dynamics must themselves be estimated from data, and "optimal" competes with several criteria rather than one. Board games strip this complexity away while preserving the structure – state, action, consequence, delayed reward – that operational decision-making shares with them. This is what makes it possible to build and rigorously test a decision-making methodology under controlled conditions before asking whether it survives the move to a less controlled, and practically consequential, environment.

Part II of this thesis carries out the first half of that programme. On Tic-Tac-Toe, Connect 4, and Othello, it implements four search algorithms of increasing sophistication – Minimax, Alpha-Beta pruning, Monte Carlo Tree Search, and Descent Minimax, an algorithm proposed by Cohen-Solal and Cazenave [1, 2] – and a convolutional evaluation network, trained by regression on millions of positions from real recorded Othello games, that is substituted as a drop-in replacement for a handcrafted evaluator across all four. Every resulting configuration is benchmarked with Wilson confidence intervals over several hundred games. Part III carries out the second half: it formalizes the Beer Game, a classical supply-chain simulation, as an MDP, states plainly how it differs from the board-game setting of Part II, and follows the developed methodology algorithm by algorithm to the point where it transfers, transfers only once a specific assumption is relaxed, or does not transfer at all.

### 1.2 Research Questions

Three questions organize this thesis. They are stated here, before any result, so that Part II and Part III can be read as evidence gathered toward answering them.

**RQ1.** Does a static, pretrained neural evaluation function improve a search algorithm's performance regardless of that algorithm's own exploration mechanism? "Static" means trained once, offline, by supervised regression on recorded outcomes, with no online refinement and no coupling to the search process that consumes it. Part II tests this across four algorithms that consume an evaluation function in structurally different ways – as a deterministic leaf evaluator (Minimax, Alpha-Beta, Descent) and as a full replacement for a stochastic rollout (MCTS).

**RQ2.** Do the search-and-evaluate methods developed on board games extend to a supply-chain optimization problem – specifically, the Beer Game, a four-echelon inventory system whose

dynamics are cooperative, stochastic, and multi-agent rather than adversarial, deterministic, and two-player? Part III tries to answer this: for each algorithm, it states whether it transfers, transfers once a specific assumption is relaxed, or does not transfer, and why.

**RQ-side.** Descent Minimax owes its effectiveness as a training-data generator, inside Cohen-Solal and Cazenave's Athénan framework, to one structural choice: every search iteration is extended to a genuine terminal state rather than stopped at a fixed depth, which is what lets Athénan train a reinforcement-learned evaluation function without a separate self-play phase [1]. The question this thesis asks is narrower, and does not itself require reinforcement learning to answer: is that same structural choice – search to a real terminal state, not to a fixed depth – advantageous on its own, even when the evaluation function it feeds is static rather than dynamic? Part II's own benchmark, run once with a handcrafted heuristic and once with a CNN trained offline, already contains the evidence needed to answer this.

### 1.3 Thesis Structure and Contribution

Part I closes with Chapter 2, which fixes the sequential decision-making framework shared by every algorithm this thesis develops, and states the theoretical grounds for the three questions above.

Part II then carries out the programme outlined in Section 1.1 on three board games of growing complexity: Tic-Tac-Toe (Chapter 3), where Minimax is introduced on a game small enough for exhaustive search; Connect 4 (Chapter 4), whose far larger state space forces depth-limited search, a handcrafted heuristic, and an empirically calibrated time budget; an unrelated image-classification task (Chapter 5), which builds the deep-learning methodology later repurposed for board evaluation; and Othello (Chapters 6 through 12), chosen for its large archive of recorded human games, where all four search algorithms are implemented, a convolutional evaluator is trained and integrated as a drop-in replacement for the handcrafted heuristic across every one of them, and the resulting twelve configurations are compared with the statistical rigor RQ1 requires.

Part III returns to RQ2 and RQ-side: Chapter 13 formalizes the Beer Game as an MDP and states how it differs from Part II's games; Chapter 14 carries out the resulting transfer analysis, algorithm by algorithm, and sketches a proposed architecture; Chapter 15 answers all three research questions in turn and closes the thesis.

The progression through Part II is deliberate: depth-limited search, a calibrated time budget, a learned evaluator, a persistent search tree – each added piece of complexity is introduced only once the simpler configuration preceding it has been implemented and tested. This turns the final, twelve-way comparison into a controlled experiment. Its result, reported with Wilson confidence intervals rather than point win rates (Chapter 12), is that the CNN evaluator improves performance over the handcrafted heuristic in every one of the four search paradigms tested – one-sidedly, with every confidence interval well above 50%, whether it acts as a deterministic leaf evaluator or replaces a stochastic rollout outright. That result answers RQ1 directly, and explains why RQ2 follows naturally: a benefit this consistent across structurally different search paradigms is a property of the evaluation function itself – exactly the kind of component more likely to transfer across problem domains than the search procedure that consumes it, which is the question Part III takes up.

### 1.4 Code and Data Availability

All the code developed for this thesis – the Tic-Tac-Toe, Connect 4 and Othello engines, the sport-classification and OthelloCNN training scripts, the benchmark harness and the analysis scripts – is publicly available, together with the raw per-game benchmark logs and the OthelloCNN training log, at `https://github.com/giacomo-sacchet/game-tree-search-cnn` [37]. The WTHOR game archive is not redistributed; the repository explains how to obtain it. File

names in the listings of this thesis refer to the development versions; the repository README maps each chapter to its script.

# 2 Theoretical Background

This chapter fixes the vocabulary shared by every algorithm this thesis develops, and states the theoretical grounds for the three questions posed in Section 1.2. Section 2.1 states the general sequential decision-making framework; Sections 2.2 through 2.5 fix the vocabulary for the four search algorithms Part II implements and benchmarks; Section 2.6 distinguishes a static from a dynamic evaluation function, the distinction RQ1 and RQ-side turn on; and Section 2.7 closes the chapter by asking which of the framework's assumptions each algorithm actually depends on – the question Chapter 14 resolves for the Beer Game specifically.

## 2.1 The Sequential Decision-Making Framework

A sequential decision-making problem is formalized as a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \pi \rangle$, where $\mathcal{S}$ is a set of states, $\mathcal{A}(s)$ the actions available in state $s$, $T(s' \mid s, a)$ a transition function giving the probability of reaching $s'$ after action $a$ in state $s$, $R(s, a, s')$ a reward, and $\pi$ a policy mapping states to actions [24, 4, 5, 6]. Given $\pi$, the value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[ \sum_{t=0}^{\infty} \gamma^{t} R(s_t, a_t, s_{t+1}) \;\middle|\; s_0 = s \right] \tag{1}$$

gives the expected cumulative reward from $s$ onward, and solving the problem means finding a policy $\pi^*$ that maximizes $V^{\pi}$ for every state – or a suitably weaker notion of optimality, where the number of decision-makers or the reward structure makes "maximizing for everyone" meaningless.

Three axes of this framework matter throughout this thesis, since Part III later argues that the Beer Game differs from Tic-Tac-Toe, Connect 4, and Othello along all three at once:

- **Deterministic versus stochastic transitions.** $T$ places all its mass on one successor state, or is a genuine distribution over several.
- **Adversarial versus cooperative objectives.** With more than one decision-maker, reward can be zero-sum – one player's gain is exactly the other's loss, the case for every game in Part II – or shared, so improving one decision-maker's outcome does not require degrading another's.
- **Single-agent versus multi-agent.** One decision-maker against a passive environment, or several whose actions jointly determine the next state.

Every board game in Part II sits at the same corner of this space: deterministic, strictly adversarial, two-player, fully observable. This is a deliberate methodological choice – it isolates the search-and-evaluate methodology from complications that a stochastic, cooperative, or many-agent setting would introduce, so the methodology can be developed and validated where it is easiest to get right, before Part III asks what happens once it is not.

## 2.2 Adversarial Search: Game Trees and Minimax

A deterministic, two-player, zero-sum, perfect-information game has a natural *game-tree* representation: the root is the current state, children are the states reachable by each legal action, and nodes alternate between the two players down to terminal states labeled by the game's outcome. *Minimax* assigns every node a value recursively – the maximum of its children's values at a maximizing node, the minimum at a minimizing node, and the outcome itself at a terminal

node – and the Minimax theorem guarantees that, searched exhaustively, this value is what both players obtain under optimal play by both sides [7].

The obstacle is combinatorial: a tree of branching factor $b$ and depth $d$ has, in the worst case, on the order of $b^d$ leaves. Tic-Tac-Toe's at most $9! = 362{,}880$ sequences (Chapter 3) sit within reach of exhaustive search; Connect 4's roughly $4.53 \times 10^{12}$ (Chapter 4) do not; Othello lies in between, tractable for exact search at moderate depth but too large to search exhaustively from an early position (Chapters 6 through 12) – which is exactly what makes it suitable for comparing search paradigms later in this thesis. Minimax's defining dependence, for the purposes of Section 2.7, is on the adversarial assumption itself: its central operation, minimizing the opponent's outcome, presupposes an opponent whose interests are exactly opposed to the searching player's.

## 2.3 Pruning and Approximation: Alpha-Beta and Depth-Limited Search

*Alpha-beta pruning* is an exact optimization of Minimax: two bounds, $\alpha$ (the best value the maximizer can already guarantee) and $\beta$ (the minimizer's equivalent), are maintained as the search proceeds, and any subtree is safely skipped once $\alpha \geq \beta$, since it cannot influence the root's value. With good move ordering this reduces the effective branching factor from $b$ to roughly $\sqrt{b}$ – a real, though not asymptotic, improvement, since the search remains exponential in depth.

Once pruning alone is not enough, *depth-limited search* becomes unavoidable: the search stops after a fixed number of plies, and every non-terminal leaf reached there is assigned a numeric estimate of its value by a *heuristic evaluation function*, designed by hand to approximate what a full search would eventually reveal. This substitution – an estimate in place of the unknown true value – is the compromise the rest of this thesis returns to repeatedly: from Connect 4's empirically calibrated "one-second limit" (Chapter 4), to the fixed search depth chosen for Othello's Minimax and Alpha-Beta (Chapter 7), to the neural evaluation function that eventually replaces the handcrafted one (Chapter 10), every one of these is a different answer to the same question – given that the position cannot be resolved exactly in the time available, what is the best available estimate of its value?

## 2.4 Sampling-Based Search: Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) estimates a position's value from an entirely different angle: rather than a fixed depth and a hand-designed heuristic, it builds an asymmetric tree by repeating four phases – selection, expansion, simulation, backpropagation – for as long as a time budget allows [23, 9]. Selection descends the tree via the UCB1 rule [22, 8],

$$\mathrm{UCB1}(s,a) = \frac{W(s,a)}{N(s,a)} + C\sqrt{\frac{\ln N(s)}{N(s,a)}}, \tag{2}$$

which balances exploiting actions with a high observed reward $W/N$ against exploring actions tried comparatively few times $N(s,a)$; expansion adds a node for an untried action; simulation (the *rollout*) estimates a new node's value, classically by playing the game out with a fast, simple policy such as uniform random play; and backpropagation updates every node on the path back to the root.

MCTS needs no heuristic evaluation function at all in its classical form – the rollout supplies its own, statistically noisy, value estimate by sampling rather than static judgement. This is also why MCTS is the algorithm in this thesis best positioned as a bridge to stochastic environments (Section 2.7): sampling a trajectory and averaging an outcome extends with little modification to a setting where the environment itself introduces randomness, a property neither Minimax nor Alpha-Beta has. Chapters 7 and 10 use the rollout first in its classical random form, then

replace it – without changing the surrounding selection, expansion, or backpropagation logic – with a single forward pass of a learned evaluation network.

## 2.5 Persistent Search and Value Refinement: Descent Minimax

Descent Minimax is the algorithm this thesis returns to most, since it is central to both RQ1 and RQ-side (Section 1.2). It originates in Cohen-Solal and Cazenave's work on learning game evaluation functions by self-play, first as a variant of *Unbounded Best-First Minimax* (UBFM) [3, 2] and later as the core search component of their Athénan framework [1]. UBFM already departs from Minimax in one respect: instead of exploring uniformly to a fixed depth, it builds a single, persistent, non-uniform tree, repeatedly extending whichever leaf lies at the end of the currently best-looking sequence of actions, re-evaluating it, and propagating the update back toward the root – which can in turn change what looks best for the next iteration.

Descent adds one change that matters a great deal: an iteration does not stop at the first unexplored leaf, as UBFM does, but continues extending the best-known line all the way to a *terminal* state before propagating values back up [2]. Two consequences follow. First, every value the search computes is exact where it counts, anchored in the game's actual outcome rather than in several layers of heuristic estimate. Second, this is precisely what makes Descent an efficient training-data generator: since no information is discarded, and no separate self-play matches are needed purely to produce learning targets, the search process is itself the data-generation process [2]. This second consequence is the premise of RQ-side: Athénan's efficiency rests on this one structural choice, searching to a real terminal state rather than a fixed depth, and the question this thesis asks is whether that choice is still an advantage once the value it feeds is a static CNN rather than a reinforcement-learned network – a question Part II's benchmark (Chapter 11) already contains the evidence to answer, since Descent is run there with both a handcrafted heuristic and a static CNN, never with reinforcement learning.

One further distinction matters. Athénan actually combines two related algorithms: Descent during *learning*, for the reasons above, and *Unbounded Best-First Minimax with Safe Decision* during *confrontations* against an opponent, which selects the move to play by visit count rather than value, trading some short-term optimality for robustness [2]. This thesis uses Descent itself, not the Safe Decision variant, as a genuine move-by-move search strategy (Chapter 8) – inheriting the search algorithm's structural property, not the learning framework built around it: a persistent partial tree, incrementally and exactly refined, that improves its estimate of the best line of play without discarding earlier work. Nothing in that operation – identify the most promising line, extend it, back up an improved estimate – has an essential dependence on the value being backed up through a strict adversarial max / min alternation, which is why Section 2.7 and Chapter 14 return to Descent as the most interesting of the four algorithms once the adversarial assumption is dropped.

## 2.6 Static and Dynamic Evaluation Functions

Every algorithm above, with the partial exception of classical MCTS, needs an evaluation function to turn a non-terminal position into a number. This thesis distinguishes two ways of obtaining one. A *handcrafted* evaluation function is a weighted combination of features a person judges predictive of the outcome – for Othello, a positional weight map favoring corners, combined with mobility, frontier exposure, and disc count (Chapter 7). A *learned* one is fitted to data instead of designed by inspection, and this thesis uses it in exactly one form throughout: a *static* network, trained once, offline, by supervised regression on a fixed corpus of recorded outcomes, then frozen and used purely for inference at every leaf a search algorithm visits. This is distinct from a *dynamic* evaluation function of the kind Athénan trains – one continually re-estimated through self-play, coupled to the search process that consumes it. RQ1 and RQ-side both turn on this distinction: RQ1 asks whether a static network, integrated as a drop-in replacement for

a handcrafted heuristic, helps regardless of the search paradigm around it; RQ-side asks whether Descent's terminal-search property is worth having even when the network it feeds is static rather than dynamic, i.e. even without the self-play loop that motivated the property in the first place. Chapter 5 develops, on an unrelated image-classification task, the practical methodology this thesis reuses to train its one static network, OthelloCNN, in Chapter 10.

### 2.7 Which Assumptions Do These Algorithms Actually Need?

Section 2.1 listed three axes along which a sequential decision-making problem can vary, and noted that every game in Part II sits at the same corner of that space. Table 1 states, for each of the four algorithms, which of the first two axes its own mechanism actually requires.

| Algorithm | Requires an adversary | Requires determinism |
|---|---|---|
| Minimax | Yes | Assumed as implemented[1] |
| Alpha-Beta | Yes | Assumed as implemented[1] |
| MCTS (UCT) | No | No |
| Descent | No | Assumed as implemented[1] |

Table 1: Whether each algorithm's own mechanism, as implemented in this thesis, depends on an adversarial reward structure and on deterministic transitions.

Minimax and Alpha-Beta are, in a precise sense, defined by the adversarial assumption: minimizing an opponent's outcome has no meaning once there is no opponent. MCTS depends on neither assumption: its backup rule can be a plain average instead of an alternating max / min, and its sampling-based simulation already handles stochastic transitions unmodified. Descent alternates max and min as implemented, but, as Section 2.5 argued, nothing about its distinguishing property – a persistent tree whose most promising line is repeatedly extended and refined – depends on that choice of backup operator. Whether that property survives contact with a genuinely cooperative, stochastic, multi-agent environment, and what becomes of Minimax, Alpha-Beta, and MCTS under the same test, is answered directly, algorithm by algorithm and with Part II's benchmark evidence in hand, in Chapter 14.

# Part II
# Search Algorithms, Neural Evaluation, and Comparison

## 3 Minimax and the Tic-Tac-Toe Implementation

### 3.1 Project Overview and Rules

The first phase of the project targets Tic-Tac-Toe (Morpion), a two-player game played on a $3 \times 3$ grid in which the first player to align three markers horizontally, vertically, or diagonally wins; if the grid fills without any alignment, the game is a draw. Beyond the game itself, this phase served as an exercise in software architecture, moving progressively from basic graphical rendering to the implementation of a full recursive decision-tree algorithm.

[1] Minimax and Alpha-Beta assume deterministic transitions as implemented here; the algorithm extends to stochastic environments via chance nodes (expectiminimax), an extension this thesis does not exercise. Descent's own backup rule alternates max and min as implemented, but nothing in its persistent, incrementally-refined structure depends on that specific backup operator, a point Chapter 14 returns to.

The implementation follows a small set of conventions that remain in force for the rest of the phase: the game state is stored in a $3 \times 3$ NumPy array (`Grille`), with 0 denoting an empty cell, 1 a cell occupied by the human player (drawn as a red cross), and 2 a cell occupied by the AI (drawn as a yellow circle). The development itself was carried out in three explicit stages: basic placement and win detection, a random AI opponent with draw detection, and finally a recursive Minimax opponent capable of perfect play.

## 3.2 Initial Architecture

Before any AI logic was introduced, the project began from a bare-bones script (`Morpion.py`) that established only the graphical environment and the grid data structure. The interface is built with `tkinter`, using a $300 \times 300$ pixel `Canvas` widget. All rendering is centralized in a single `Dessine()` function, which redraws the grid lines and iterates over the `Grille` array to place a cross or a circle in each occupied cell. User interaction is handled by a `MouseClick` callback bound to the left mouse button: pixel coordinates are converted to grid indices by integer division by 100, and the corresponding cell is updated.

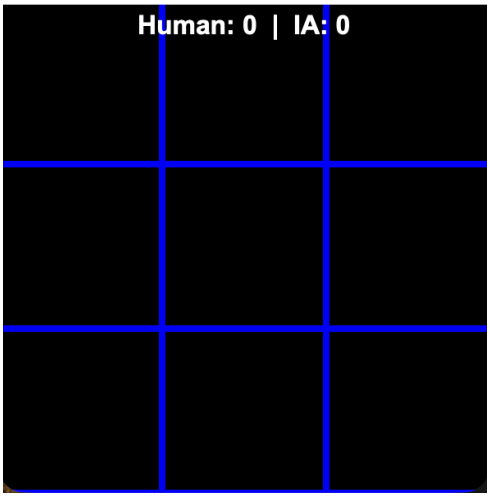



Figure 1: The initial `tkinter` canvas: a $3 \times 3$ grid with a live human/IA score readout.

This starting point, however, provided only a drawing board: it contained no win-detection logic, no turn alternation, no protection against clicking an already-occupied cell, and no AI opponent at all. Every one of these gaps is addressed over the following two development stages.

## 3.3 Evolution of the AI Player

### 3.3.1 A Random Baseline

The first working version of the AI, introduced together with formal win detection, adds a `WinnerCheck()` function that uses NumPy slicing (`np.all` over rows, columns, and both diagonals) to detect an alignment of three identical markers, returning a code that distinguishes a human win, an AI win, a draw, and an ongoing game. The AI itself, `PlayAI()`, does nothing more sophisticated than collecting the coordinates of all empty cells with `np.where(Grille == 0)` and picking one uniformly at random with `random.choice`. Simple as it is, this baseline is essential: it validates the surrounding game logic (turn sequencing, win detection, board reset) before any strategic behaviour is layered on top, and it establishes the visual feedback convention used throughout the rest of the phase — the grid colour turns red on a human win, yellow on an AI win, and white on a draw.

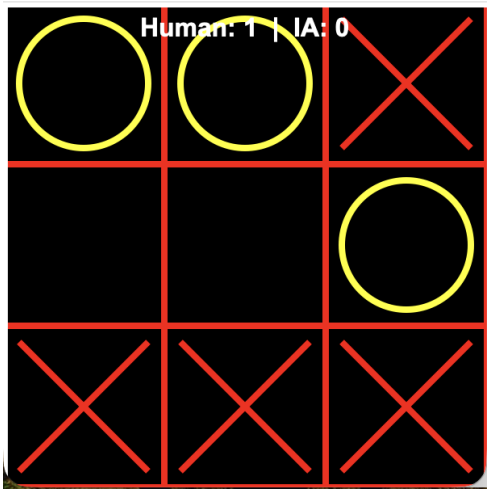



Figure 2: End-of-match state: the grid border turns red to signal a human victory, while the current score is kept on-screen.

### 3.3.2 First Recursive Iteration

The second stage replaces random play with a genuine game-tree search. Two mutually recursive functions, `SimulIA()` and `SimulHuman()`, simulate the game to its end from the current position: each one checks for a terminal state via `WinnerCheck()`, and if the game is not yet over, it tries every empty cell in turn — playing the move, recursing into the opponent's simulation function, and then undoing the move (backtracking) before trying the next candidate. `SimulIA()` favours the ordering IA win $>$ draw $>$ human win, while `SimulHuman()` assumes a perfectly rational human who favours human win $>$ draw $>$ IA win.

In this first version, the recursive functions return only the predicted outcome, not the coordinates that produce it. As a consequence, `PlayAI()` has to re-implement the first level of the search itself: it loops over the empty cells of the real board, simulates each one, and picks the cell whose simulated outcome has the highest priority. This duplication — searching once inside `PlayAI()` and again inside the recursive functions — is the main architectural weakness addressed in the next iteration.

### 3.3.3 Final Refactor: Tuple Return Values

The final version of the plain Minimax implementation removes this duplication by having `SimulIA()` and `SimulHuman()` return a tuple `(best_cell, best_result)` instead of the outcome alone. The best move is now tracked directly inside the recursion as it explores the tree, and terminal states simply return `None` for the cell together with the corresponding result string. This lets `PlayAI()` shrink to a single call to `SimulIA()` followed by applying the returned move to the real board, cleanly separating the search engine from the game loop.

## 3.4 The Minimax Algorithm: Logic and Implementation

Conceptually, Minimax explores the entire game tree from the current position: at every AI turn it picks the move that maximizes the AI's outcome, and at every human turn it assumes the opponent will pick the move that is best for them (which is equivalent to minimizing the AI's outcome), until every branch terminates in a win, a loss, or a draw. Because Tic-Tac-Toe's game tree is small enough to explore exhaustively (at most $9! = 362{,}880$ move sequences), this search is guaranteed to be optimal: the AI implemented this way cannot be beaten, and will win whenever the opponent gives it the chance to.

Two properties of this implementation are worth emphasizing. First, the algorithm is stateless with respect to history: it makes no use of the human's past behaviour, deciding purely from the current board configuration and the moves still available. Second, exploration is achieved through backtracking rather than through board copies: each simulation function temporarily writes a move into the shared `Grille` array, recurses, and then restores the cell to empty, which keeps memory use constant regardless of search depth.

Terminal outcomes are compared through a simple priority mapping, `{"IA": 3, "N": 2, "H": 1}` from the AI's perspective (and the mirrored mapping from the human's), so that a winning line is always preferred to a draw, which is in turn always preferred to a loss.

### 3.5 Depth Optimization in Minimax

Although the plain Minimax opponent is unbeatable, it is indifferent to *how quickly* it wins or loses: a move that wins in five turns is treated as equal to a move that wins immediately, and a forced loss in one move is treated as equal to a forced loss delayed as long as possible. The final iteration of this phase, implemented in `TTT_Minimax_depth.py`, fixes this by threading a `depth` counter through the recursion, incremented by one at every recursive call, and returned alongside the result.

This extra piece of information refines the move-selection rule: among several moves that all lead to an AI win, the one with the *lowest* depth is chosen, so the AI always takes the fastest path to victory; among several moves that all lead to a forced loss, the one with the *highest* depth is chosen instead, maximizing the chance that the human makes a mistake along the way; and among several moves leading to a draw, the lowest depth is again preferred, for an efficient, non-dithering game. Listing 1 shows the final depth-aware version of `SimulIA()`, which is structurally identical to `SimulHuman()` with the priority mapping mirrored.

```
def SimulIA(depth=0):
    # 1. Terminal check, returning depth alongside the result
    res = WinnerCheck()
    if res == 2: return None, "IA", depth
    if res == 1: return None, "H", depth
    if res == 3: return None, "N", depth

    empty_cells = list(zip(*np.where(Grille == 0)))
    Results = []

    for (x, y) in empty_cells:
        Grille[x][y] = 2                        # Play the move
        _, R, D = SimulHuman(depth + 1)         # Recurse, depth + 1
        Results.append(((x, y), R, D))
        Grille[x][y] = 0                        # Undo (backtrack)

    priority_ia = {"IA": 3, "N": 2, "H": 1}
    best_cell, best_result, best_depth = Results[0]

    for cell, result, d in Results[1:]:
        if priority_ia[result] > priority_ia[best_result]:
            best_result, best_cell, best_depth = result, cell, d
        elif priority_ia[result] == priority_ia[best_result]:
            if result == "IA" and d < best_depth:        # win faster
                best_result, best_cell, best_depth = result, cell, d
            elif result == "H" and d > best_depth:        # lose slower
                best_result, best_cell, best_depth = result, cell, d
            elif result == "N" and d < best_depth:        # draw faster
                best_result, best_cell, best_depth = result, cell, d

    return best_cell, best_result, best_depth
```

Listing 1: Depth-aware recursive Minimax search for the AI player (`TTT_Minimax_depth.py`).

### 3.6 Conclusions

The Tic-Tac-Toe phase carried the project from a bare graphical template to a fully unbeatable AI opponent, through three clearly delineated stages: establishing rules and win detection, introducing a random baseline, and finally deploying a recursive Minimax search. The refactor from outcome-only to tuple-returning simulation functions is a useful architectural lesson in its own right, since it collapses two redundant search loops into one and cleanly separates search from game control — a separation that recurs, in more elaborate form, in every later phase of the project. The addition of a depth counter, while a small code change, turns a merely correct AI into one that plays with a clear and intuitive sense of urgency, a property that later phases (Connect 4, Othello) preserve through explicit search-depth limits and, eventually, through statistical evaluation of move quality.

## 4 Connect 4: Depth-Limited Minimax and the 3-Player Extension

### 4.1 Introduction

The second phase of the project transposes Connect 4 (Forza 4) to a digital environment, moving from a basic playable implementation to a system offering multiple, directly comparable levels of AI. The game is built on the same technology stack as Tic-Tac-Toe — Python, NumPy for grid state, and `tkinter` for the interface — but its far larger search space forces the introduction of concepts that Tic-Tac-Toe never required: heuristic leaf evaluation, depth-limited search, and empirical, hardware-aware performance tuning. The phase closes with an original extension of the game to a three-player format, which exercises the generality of the architecture beyond the two-player case for which it was designed.

### 4.2 From Tic-Tac-Toe to Connect 4: Core Architecture

Adapting the Tic-Tac-Toe code base to Connect 4 required three structural changes. First, the grid grew from $3 \times 3$ to $6 \times 7$. Second, and more importantly, Connect 4 replaces free cell placement with a physically constrained "drop" mechanic: a click selects a column, not a cell, and a new helper, `FirstFreeRow`, scans that column from the bottom row upward to find the lowest empty slot, emulating gravity. Third, `WinnerCheck` had to be entirely rewritten: instead of scanning for three aligned marks, it now scans the $6 \times 7$ board for four consecutive tokens of the same colour along rows, columns, and both diagonal directions. Together, these changes move the architecture from a coordinate-based placement system to a column-based drop system that mirrors the physical game.

### 4.3 AI Level 1: Random Baseline

As in the Tic-Tac-Toe phase, the first AI is a purely stochastic agent: `GetPossibleMoves` lists every column whose top row is still empty, and `PlayAI` picks one with `random.choice`, relying on the gravity logic to determine the actual landing row. Beyond confirming that the core rules (victory detection, draw conditions) behave correctly, this random agent is kept throughout the project as a fixed benchmark: every more advanced strategy is first validated by measuring its win rate against random play, a pattern that becomes the backbone of the statistical testing methodology used later in the Othello phase (Chapter 11).

### 4.4 AI Level 2: Heuristics and Informed Scores

The second AI evaluates the immediate consequence of each candidate move rather than choosing blindly. Its core primitive, `GetMaxAlignment`, simulates a move and scans all four directions

to find the largest run of same-colour tokens it would create inside any window of four slots. Candidate moves are then ranked by a fixed priority ladder, where only the single highest-priority rule matched applies:

- **100 points** — the move completes a four-in-a-row (immediate win).
- **50 points** — the move blocks an opponent's immediate four-in-a-row.
- **30 points** — the move creates an alignment of three out of four.
- **15 points** — the move blocks an opponent's three-out-of-four.
- **10 points** — the move forms a basic alignment of two out of four.

A dedicated simulation engine pits this heuristic agent against the random baseline over many games; the heuristic AI shows a consistently dominant win rate, confirming that prioritizing immediate threats and opportunities is already a substantial improvement over random play.

## 4.5 AI Level 3: The Minimax Algorithm

### 4.5.1 Theory and Complexity

Connect 4 cannot be searched exhaustively the way Tic-Tac-Toe can. Where Tic-Tac-Toe has at most $9! = 362{,}880$ possible move sequences, Connect 4 admits up to 42 plies and on the order of $4.531 \times 10^{12}$ (4.5 trillion) distinct possible games — roughly ten million times more than Tic-Tac-Toe. A full-depth Minimax search is therefore computationally out of reach, and the project instead adopts *depth-limited* Minimax: the search explores only a fixed number of plies ahead rather than to the end of the game.

### 4.5.2 Limited-Depth Minimax and Grid Evaluation

The `Minimax` function recurses to a fixed depth, alternating between a maximizing layer (the AI's turn) and a minimizing layer (the opponent's turn) via an `is_maximizing` flag. Because the recursion is cut off before most branches reach a terminal state, every non-terminal leaf needs a numeric estimate of how good the position is; this is the role of `EvaluateGrid`, which returns $+500$ for an immediate AI win, $-500$ for an immediate opponent win, and otherwise the difference between the AI's and the opponent's best positional score (`max_ia - max_human`), reusing the same weighted heuristic table introduced in the previous section. Listing 2 shows the recursive core together with the evaluation function it calls at the leaves.

```
def EvaluateGrid(p_id, opp_id):
    state = CheckGameState()
    if state == p_id:   return 500    # AI wins
    if state == opp_id: return -500   # Opponent wins
    if state == 4:       return 0     # Draw

    p_scores, opp_scores = [], []
    for col in range(7):
        row = FirstFreeRow(col)
        if row != -1:
            p_scores.append(GetScoreForPosition(row, col, p_id))
            opp_scores.append(GetScoreForPosition(row, col, opp_id))

    max_p   = max(p_scores) if p_scores else 0
    max_opp = max(opp_scores) if opp_scores else 0
    return max_p - max_opp   # AI's positional advantage


def Minimax(depth, is_maximizing, p_id, opp_id):
    state = CheckGameState()
    if depth == 0 or state != 0:
        return EvaluateGrid(p_id, opp_id)

    possible_moves = GetPossibleMoves()
```

```
    if is_maximizing:
        best_val = -float('inf')
        for col in possible_moves:
            row = FirstFreeRow(col)
            Grid[row][col] = p_id
            best_val = max(best_val, Minimax(depth - 1, False, p_id, opp_id))
            Grid[row][col] = 0
        return best_val
    else:
        best_val = float('inf')
        for col in possible_moves:
            row = FirstFreeRow(col)
            Grid[row][col] = opp_id
            best_val = min(best_val, Minimax(depth - 1, True, p_id, opp_id))
            Grid[row][col] = 0
        return best_val
```

Listing 2: Depth-limited Minimax and leaf evaluation for Connect 4 (`Connect4_final_Heuristic_Athenan.py`).

With the search depth limited to only 3 plies to keep the AI responsive, its measured performance is close to that of the plain heuristic agent: since both use the same scoring table once the depth limit is hit, and effective Connect 4 tactics often require anticipating traps 5–6 moves ahead, the Minimax agent behaves similarly to the heuristic one at this depth, but at markedly higher computational cost. This observation motivates the empirical performance study described next.

## 4.6 Optimization and Performance Testing

### 4.6.1 Empirical Analysis of Calculation Times

To turn "depth-limited" into a principled engineering choice rather than an arbitrary constant, a dedicated `TestDepthPerformance` routine measures, for a neutral empty board, how long `PlayAIMinimax` takes to choose a move at increasing depths (typically 1 through 9), resetting the grid between measurements to keep conditions comparable.

### 4.6.2 The "One-Second Limit"

The explicit target of this test is the deepest search that still responds in under one second, since exceeding this threshold visibly breaks the fluidity of an interactive game. As soon as a given depth $d$ exceeds the 1.0-second budget, the routine reports $d-1$ as the recommended maximum depth for the current hardware and stops (Listing 3). This empirical, hardware-aware tuning step is the direct ancestor of the fixed time budgets (rather than fixed depths) used for MCTS and Descent in the later Othello phase.

```
def TestDepthPerformance():
    global Grid
    Grid = np.zeros((6, 7), dtype=int)
    p_id = 2
    for d in range(1, 10):
        start_time = time.time()
        AI_FUNCTIONS["3"](player_id=p_id, depth=d)
        duration = time.time() - start_time
        Grid = np.zeros((6, 7), dtype=int)
        print(f"Depth {d}: {duration:.4f} seconds")
        if duration > 1.0:
            print(f"RESULT: recommended max depth is {d-1}.")
            break
```

Listing 3: Empirical search for the maximum Minimax depth under a 1-second budget.

### 4.7 Game Extension: 3-Player Mode

#### 4.7.1 An Alternative Victory Condition

The final extension of this phase introduces a third participant, the Green AI, alongside the Human and the Red AI. Rather than competing for a standard four-in-a-row, Green wins by forming a $2 \times 2$ square of its own tokens anywhere on the board — a fundamentally different geometric target that required its own detection routine, scanning every $2 \times 2$ window of the $6 \times 7$ grid for four green tokens, and its own positional scoring helper, `GetSquareScore`, which evaluates how much a candidate move contributes to completing such a square.

#### 4.7.2 Systemic Modifications and Current AI Strategies

Supporting a third player required rotating turns as Human → Red AI → Green AI inside the `MouseClick` event handler, and extending `Dessine` to render green tokens alongside a three-way scoreboard. In the current implementation, both AIs (Red and Green) use independent heuristic evaluations tailored to their respective goals, and neither cooperates with nor actively targets the other — each plays against the human in isolation. A natural extension, noted but not implemented within this phase, would generalize Minimax itself so that every node returns a three-element score vector [27] (one entry per player), with each player maximizing their own component during back-propagation; the added branching factor this implies would, however, make the one-second responsiveness budget considerably harder to maintain.

### 4.8 Final Structure and Conclusions

The finished program is organized around a single `__main__` entry point offering four operational modes — Play vs AI, AI vs AI Simulation, Empirical Depth Test, and the 3-Player Mode — with an `AI_FUNCTIONS` dictionary mapping user choices to the underlying strategy functions, letting the engine swap between Random, Heuristic, and Minimax behaviour without touching the core game loop. This dispatcher pattern, introduced here, is carried forward essentially unchanged into the Othello engine described in Chapter 6.

Overall, the Connect 4 phase successfully carries the project from a $3\times3$, exhaustively solvable game to a $6 \times 7$ environment whose state space is vast enough to require depth-limited search, heuristic leaf evaluation, and hardware-calibrated time budgeting — the same three ingredients that reappear, considerably more elaborated, throughout the Othello phase.

## 5 Training a CNN for Sport Image Classification

### 5.1 Overview: Two Parallel Tracks

This phase of the project temporarily leaves board games aside to develop and train convolutional neural networks (CNNs) for an unrelated task: classifying images into eight sport categories. The detour is deliberate. Building a CNN for a well-understood, easily-evaluated task (image classification, with abundant labelled data) makes it possible to systematically practice the full toolbox of modern deep learning — architecture design, optimizer choice, regularization, learning-rate scheduling, data augmentation, and transfer learning — before that toolbox is redeployed, in Chapter 10, to train a very different kind of network: one that evaluates Othello board positions rather than classifying photographs.

Two distinct approaches are pursued, deliberately using two different frameworks so that both ecosystems are exercised:

- **TinyCNN Sport** — a small custom architecture built and iteratively refined from scratch in **PyTorch**, evolving through seven successive experiments (optimizer, learning rate, normalization, scheduling, augmentation, and finally a deeper architecture).

- **Transfer learning with MobileNetV2** — a pretrained convolutional backbone adapted to the sport domain in **Keras/TensorFlow**, following the progressive three-phase methodology (feature extraction, feature extraction with augmentation, fine-tuning) described in Chollet's *Deep Learning with Python* [21].

## 5.2 The Dataset

Both tracks share the same dataset: eight sport categories — football (soccer), tennis, basketball, volleyball, boxing, cycling, golf, and Moto GP — with a target of 1,500 images per class. The category list was refined during data collection: swimming was dropped in favour of boxing (too easily separated by colour alone to be a challenging class), running was dropped in favour of cycling (only 1,300 images were available, short of the 1,500 target), and climbing was dropped in favour of Moto GP (climbing images were too visually homogeneous across the class). Roughly 1,600 JPEG images per category were downloaded and then manually pruned to remove unsuitable examples, a quality-control pass that took from about 10 minutes (football) to 30 minutes (tennis) per category.

## 5.3 TinyCNN Sport: Iterative Refinement in PyTorch

The from-scratch track starts from a minimal convolutional network trained on $64 \times 64$ images and improves it through seven controlled experiments, each isolating a single change and building directly on the best configuration found previously.

### 5.3.1 Baseline: Adam at LR $= 10^{-3}$

The first experiment simply establishes a working baseline: Adam optimizer [18], learning rate $10^{-3}$ (Adam's standard starting point), 20 epochs. The model reaches a training accuracy of 94.2% and a validation accuracy of 88.8%, with a 5.4-point train/validation gap indicating the onset of overfitting; the validation loss bottoms out at epoch 14 and oscillates afterward, suggesting epoch 15 as a natural early-stopping point. The average Softmax confidence on the predicted class (91.8%) exceeding the actual validation accuracy further signals mild overconfidence.

### 5.3.2 Learning Rate Sweep

To find the right step size, five learning rates spanning four orders of magnitude ($10^{-1}$ to $10^{-5}$) are compared under otherwise identical conditions (Figure 3). The results are unambiguous: $10^{-1}$ fails outright (accuracy stuck at $\approx$12.5%, equivalent to random guessing over 8 classes, because the optimizer overshoots every minimum); $10^{-2}$ learns but oscillates strongly; $10^{-3}$ is the clear winner, exceeding 80% accuracy by epoch 10 and peaking at 88.7% validation accuracy within 20 epochs; $10^{-4}$ is smooth and stable but too slow to catch up within the epoch budget; and $10^{-5}$ is prohibitively slow, reaching only $\approx$60% accuracy.

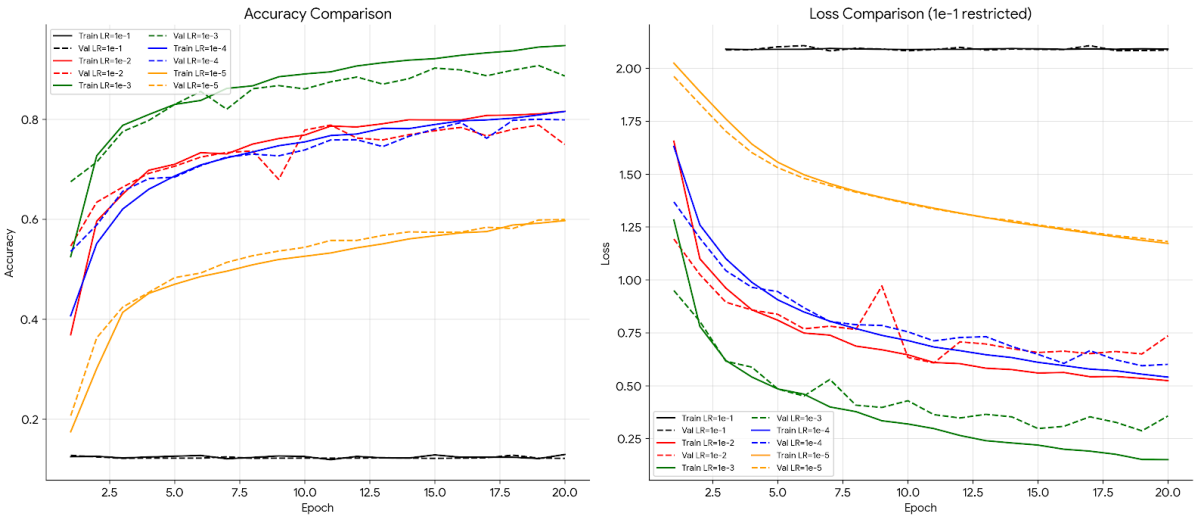



Figure 3: Accuracy and loss for five learning rates ($10^{-1}$ to $10^{-5}$), Adam optimizer, 20 epochs. LR = $10^{-3}$ (green) dominates within the epoch budget.

#### 5.3.3 SGD with Momentum

Keeping LR = $10^{-3}$ as the reference, the next experiment replaces Adam with Stochastic Gradient Descent (momentum 0.9, weight decay $10^{-4}$), extended to 40 epochs to give the optimizer time to converge. SGD ultimately reaches a peak validation accuracy of 92.23% at epoch 38, clearly surpassing Adam's 88.80% peak, and drives training accuracy to a near-perfect 98.5%, though a late dip to 89.59% at the very last epoch hints that the fixed learning rate becomes too large once the model nears its optimum.

#### 5.3.4 Batch Normalization

Adding Batch Normalization (BN) [16] after each convolution stabilizes the internal activation statistics and produces an immediate "initial leap": first-epoch validation accuracy jumps from 54.1% (plain SGD) to 76.4% with BN, crossing the 80% threshold a full epoch earlier. The BN model also edges out the peak accuracy (92.48% vs. 92.23%) and, importantly, ends training more stably, at 91.39% versus 89.59% for the non-BN model — at the cost of occasional sharp validation-loss spikes attributable to batch-wise statistics.

#### 5.3.5 Learning Rate Scheduler

To eliminate the remaining late-stage instability, a three-stage step scheduler is introduced: LR = $10^{-2}$ for epochs 1–20, $10^{-3}$ for epochs 21–35, and $10^{-4}$ for epochs 36–40. This raises peak validation accuracy to 93.98% (from 92.48%) and, more importantly, collapses the standard deviation of validation accuracy after epoch 20 from ≈0.025 to ≈0.003 — the scheduler does not just improve the peak, it removes the noise around it.

#### 5.3.6 Data Augmentation

Random crops and horizontal flips are then added to the training pipeline. The effect is the most qualitatively significant of the whole sequence: not only does validation accuracy reach a new peak of 94.78%, but the persistent ≈5-point train/validation gap seen in every previous experiment essentially disappears, with validation accuracy (94.78%) even slightly exceeding training accuracy (94.13%) — an expected artefact of training on harder, artificially distorted images while validating on clean ones.

### 5.3.7 Better CNN: A Double Convolutional Architecture

The final architectural change deepens each convolutional stage from one to two stacked $3 \times 3$ convolutions [17] (each followed by Batch Normalization and ReLU) before max-pooling, giving the network more capacity to learn higher-order, compositional patterns. Combined with all previous improvements (SGD with momentum, Batch Normalization, the step scheduler, and data augmentation), this configuration reaches a final validation accuracy of **96.20%**, the best of the entire sequence, while keeping the training/validation gap to a controlled 1.4% (Figure 4). Listing 4 shows the resulting building block and overall architecture.

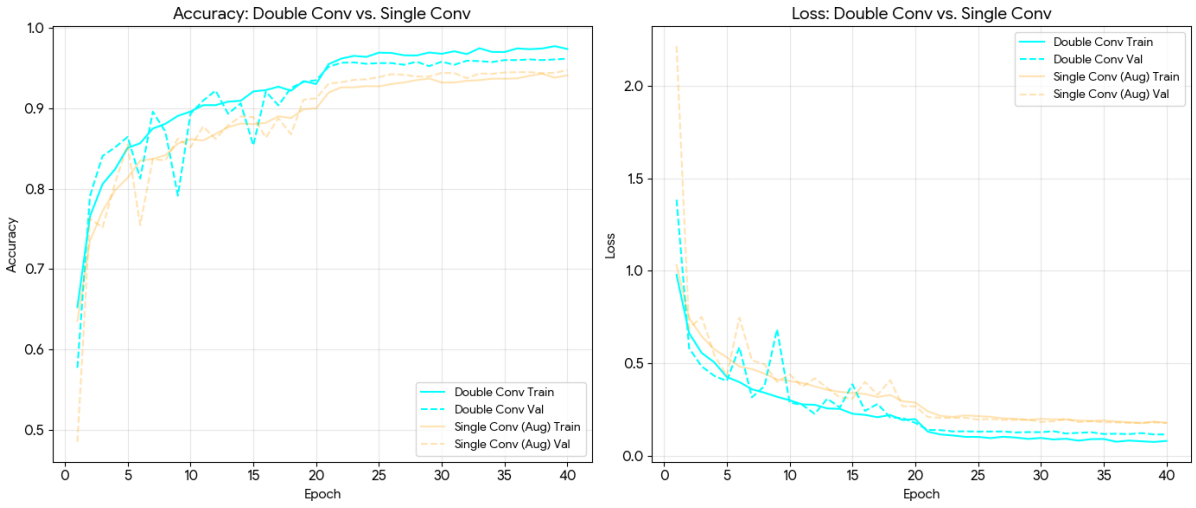



Figure 4: Double-convolution architecture vs. the previous single-convolution model with augmentation: accuracy and loss over 40 epochs.

```python
class ConvBlock(nn.Sequential):
    def __init__(self, c_in, c_out):
        super().__init__(
            nn.Conv2d(c_in, c_out, 3, padding=1),
            nn.BatchNorm2d(c_out),
            nn.ReLU(inplace=True),
            nn.Conv2d(c_out, c_out, 3, padding=1),
            nn.BatchNorm2d(c_out),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2)
        )

class CifarCNNBase(nn.Module):
    def __init__(self, num_classes=8):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(3, 64),       # L -> L/2
            ConvBlock(64, 128),     # L/2 -> L/4
            ConvBlock(128, 256),    # L/4 -> L/8
        )
        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

Listing 4: Double-convolution building block and final TinyCNN Sport architecture (PyTorch).

Table 2 summarizes the cumulative effect of each optimization across the seven experiments.

| Configuration | Peak Val. Accuracy | Notes |
|---|---|---|
| Adam, LR $= 10^{-3}$ (baseline) | 88.8% | 5.4 pt overfitting gap |
| SGD + momentum | 92.23% | Late-epoch instability |
| + Batch Normalization | 92.48% | Faster initial convergence |
| + LR Scheduler | 93.98% | Noise reduced 8× |
| + Data Augmentation | 94.78% | Overfitting gap eliminated |
| + Double Convolution (final) | **96.20%** | 1.4 pt gap, most confident |

Table 2: Cumulative validation accuracy across the TinyCNN Sport experiment sequence.

### 5.4 Transfer Learning with MobileNetV2

The second track asks a different question: rather than engineering a small network from scratch, how much can be gained by adapting a large network already pretrained on ImageNet [20]? This track uses MobileNetV2 [19] as a frozen (and later partially unfrozen) feature extractor, implemented in Keras/TensorFlow and run on Google Colab with GPU acceleration, following the same 12,000-image dataset (1,500 per class, 80/20 train/validation split, fixed seed 42).

#### 5.4.1 Preprocessing and Architecture

Images are resized to $224 \times 224$, MobileNetV2's native resolution, and rescaled to $[-1, 1]$ via `mobilenet_v2.preprocess_input`. For Phases 2 and 3, on-the-fly augmentation layers (`RandomFlip`, `RandomRotation(0.1)`, `RandomZoom(0.2)`) are applied only during training. The shared classifier head is a `GlobalAveragePooling2D` layer, a `Dense(256, relu)` layer, `Dropout(0.5)`, and a final `Dense(8, softmax)`. Throughout all three phases, MobileNetV2's BatchNormalization layers are explicitly kept in inference mode, preserving the running statistics learned on ImageNet rather than corrupting them with small-batch statistics from the sport dataset.

#### 5.4.2 Phase 1: Fast Feature Extraction

In Phase 1, the frozen MobileNetV2 base is run once over the entire dataset to precompute a $(N, 7, 7, 1280)$ feature array, and only the Dense head is trained on these cached features for 20 epochs — no augmentation is possible here since each image is seen exactly once. Training is extremely fast, and validation accuracy already exceeds 97% at epoch 1, peaking at **98.54%** at epoch 18.

#### 5.4.3 Phase 2: Feature Extraction with Augmentation

Phase 2 assembles an end-to-end model (augmentation → frozen MobileNetV2 → Dense head) and trains for 50 epochs. Because every image now passes through the full convolutional base at every step, training is substantially slower, but the wider input distribution acts as a regularizer: the train/validation gap tightens, even though peak validation accuracy (98.45%) does not improve on Phase 1.

#### 5.4.4 Phase 3: Fine-Tuning

Phase 3 starts from the best Phase 2 checkpoint and unfreezes only the last 30 layers of MobileNetV2 (all BatchNormalization layers remain frozen regardless), then jointly re-trains these layers with the Dense head at a very low learning rate ($10^{-5}$) for 30 more epochs. Listing 5 shows

the unfreezing logic. This produces the best result of the three phases, **98.83%** validation accuracy at epoch 26 (Figure 5), confirming that even high-level, ImageNet-derived representations can be beneficially specialized to a narrower domain.

```
conv_base = model.get_layer('mobilenetv2_1.00_224')
conv_base.trainable = True

# Keep all layers before the last 30 frozen
for layer in conv_base.layers[:-30]:
    layer.trainable = False

# BatchNormalization layers always stay in inference mode
for layer in conv_base.layers:
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False

model.compile(
    loss=keras.losses.SparseCategoricalCrossentropy(),
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-5),
    metrics=['accuracy']
)
```

Listing 5: Partial unfreezing of MobileNetV2 for fine-tuning (Keras/TensorFlow).

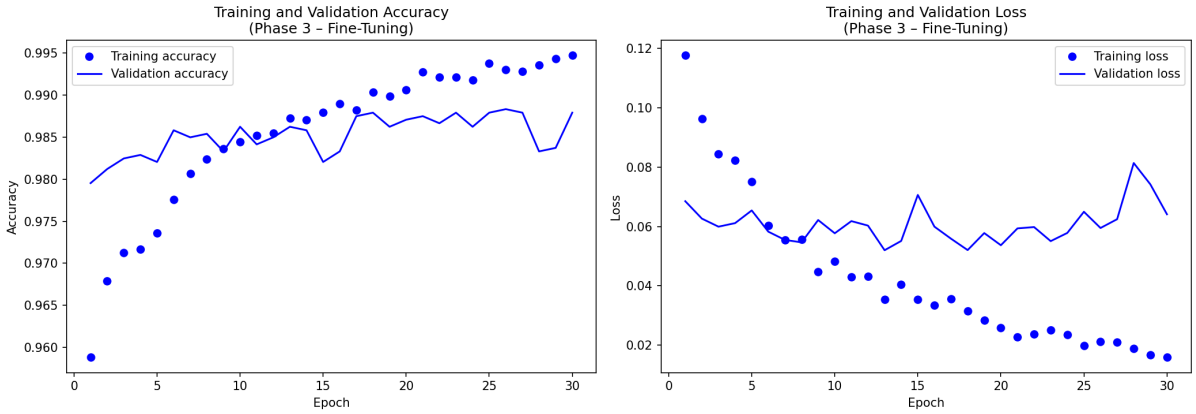



Figure 5: Phase 3 fine-tuning: training and validation accuracy and loss over 30 epochs, starting from the Phase 2 checkpoint.

### 5.4.5 Cross-Phase Comparison

| Phase | Epochs | Best Val. Accuracy | Best Val. Loss |
|---|---|---|---|
| 1 — Fast feature extraction (no aug.) | 20 | 98.54% | 0.0520 |
| 2 — Feature extraction (with aug.) | 50 | 98.45% | 0.0829 |
| 3 — Fine-tuning (from Phase 2) | 30 | **98.83%** | 0.0520 |

Table 3: Transfer learning results by phase.

All three phases exceed 98% validation accuracy, showing that MobileNetV2's ImageNet features transfer remarkably well to the sport domain even without any fine-tuning at all. Phase 1 is the most computationally efficient and is recommended when resources are limited; Phase 2 offers stronger regularization for smaller or longer-trained datasets; Phase 3 delivers the best absolute accuracy when GPU budget allows, though the marginal gain over Phase 2 (+0.38 points) is

modest given the already high quality of the pretrained features and the reasonably large size of the dataset.

### 5.5 Discussion: Two Frameworks, One Set of Lessons

It is worth being explicit that these two tracks are not the same network reused twice: TinyCNN Sport is a genuinely custom architecture trained from randomly initialized weights in PyTorch, while the MobileNetV2 pipeline in Keras/TensorFlow starts from ImageNet-pretrained weights and only ever trains a small Dense head plus, in Phase 3, a handful of unfrozen convolutional layers. What carries over from this phase to the rest of the project is not the network itself but the underlying methodology — systematic, single-variable-at-a-time experimentation; the value of normalization and scheduling for training stability; the regularizing effect of data augmentation; and, from the transfer-learning track specifically, direct experience with freezing, partial unfreezing, and very-low-learning-rate fine-tuning. Chapter 10 reuses exactly this methodology, in PyTorch, to train OthelloCNN — a residual network trained by regression, from scratch, on millions of positions extracted from real Othello games.

## 6 From Image Classification to Game Evaluation: Transitioning to Othello

### 6.1 From Classifying Images to Evaluating Boards

Chapter 5 established, on an unrelated image-classification task, that a convolutional network can be trained to turn raw pixel data into a single, reliable numeric judgement — a probability distribution over eight sport classes. Once that capability is in hand, a natural question follows: could the same kind of network be trained to turn a *board position* into a single, reliable numeric judgement of how good that position is for a given player? If so, a CNN could replace the handcrafted evaluation functions (`EvaluateGrid` in Tic-Tac-Toe and Connect 4) that every search algorithm so far has relied on at its leaves.

This is not a hypothetical extension. It is the central idea of the fifth phase of the project, and it is the reason the game itself changes at this point. Realizing it requires two ingredients that Connect 4 cannot supply: enough labelled board positions to train a network by supervised learning, and a domain where a positional evaluation is genuinely non-obvious enough that learning it is worthwhile. Both requirements point away from Connect 4 and toward Othello.

### 6.2 Why Othello? The WTHOR Dataset Advantage

#### 6.2.1 Limitations of Connect 4 for Supervised Learning

Connect 4 is a fully solved game: with optimal play, the first player always wins. This is precisely what makes it unsuitable as a training ground for a value network. Training a CNN evaluator by supervised learning requires two things: a sufficiently large corpus of board positions, and a reliable ground-truth signal for each one (typically, the eventual outcome of the game it came from). No standard, large-scale, machine-readable archive of recorded Connect 4 games exists at the scale needed for this — the previous phase's Minimax and heuristic agents were never validated against a corpus of real, human-played games, only against each other.

#### 6.2.2 The WTHOR Format

Othello does not share this limitation. The French Othello Federation maintains a large, publicly available archive of competitive games, distributed through the Cassio game engine in the WTHOR binary format (`.wtb` files). Each file begins with a 16-byte header (including the game count) followed by a sequence of 68-byte game records; within each record, the final Black disc

count is stored at byte offset 6, and the 60 move slots begin at offset 8, with each move encoded as a single byte, `(col+1)*10 + (row+1)`, in 1-based coordinates. Parsing and replaying every game move by move, then extracting every intermediate position from move 10 onward (discarding the highly stereotyped opening) and pairing it with the game's final disc margin, yields on the order of several million labelled training samples from a moderate-sized archive — exactly the kind of dataset that Connect 4 cannot provide.

### 6.2.3 Othello's Structural Properties

Beyond sheer data availability, Othello has properties that make it particularly well suited to neural evaluation:

- **Fixed board size.** The board is exactly $8 \times 8$, small enough for a CNN with no pooling to process at high speed without losing spatial resolution.
- **Dihedral symmetry.** Every position has a full symmetry group of order 8 (the dihedral group $D_4$: four rotations, each with a mirror reflection), enabling eightfold data augmentation at no additional labelling cost.
- **Non-trivial positional evaluation.** Disc count is often a misleading signal early in the game, while corner control and mobility are strategically decisive — making the evaluation function genuinely non-linear and worth learning rather than hand-tuning [26].
- **Tractable search space.** Unlike Chess or Go, Othello's state space remains small enough for exact Minimax search at moderate depths, allowing a direct, controlled comparison between pure search, pure heuristic, and hybrid CNN-guided approaches — the comparison that occupies the rest of this report.

## 6.3 Code Architecture: A Five-Block Layout

The change of game is accompanied by a deliberate reorganization of the code base, designed to keep the project's phases architecturally comparable even as their content diverges. The Othello engine follows the same five-block template first established in the Connect 4 phase:

1. **Globals and constants** — the $8 \times 8$ board array (`Grid`, with 0 empty, 1 Black/Human, 2 White/AI), score counters, turn state, and shared constants such as the positional weight map used later by the heuristic evaluator.
2. **Game logic** — move legality, disc-flipping, turn management, and terminal-state detection.
3. **3a. CNN backend** — the neural evaluator: architecture, board encoding, and inference (Chapter 10).
4. **3b. AI strategies** — the six search algorithms compared in this report (Chapters 7 and 8), each registered under a common dispatcher.
5. **Graphical interface** — the `tkinter` rendering and interaction layer, structurally similar to the Connect 4 GUI but adapted to an $8 \times 8$ board.
6. **Simulation and execution control** — batch AI-vs-AI play, benchmarking, and the top-level menu (Chapter 11).

Listing 6 shows the core of Block 1: the board representation and the positional weight map that the handcrafted heuristic (introduced in the next chapter) reuses at every leaf evaluation.

```
# Initialize empty 8x8 grid (0 = empty, 1 = Black/Human, 2 = White/AI)
Grid = np.zeros((8, 8), dtype=int)

# Positional weight map for the 8x8 board.
# Corners are the highest-value cells; X-squares (diagonally adjacent to
# corners) are the most dangerous to occupy, since they hand corners
# to the opponent.
POSITION_MAP = np.array([
    [ 100, -20,  10,   5,   5,  10, -20,  100],
    [ -20, -40,  -5,  -5,  -5,  -5, -40,  -20],
```

```
    [  10,  -5,   3,   1,   1,   3,  -5,   10],
    [   5,  -5,   1,   0,   0,   1,  -5,    5],
    [   5,  -5,   1,   0,   0,   1,  -5,    5],
    [  10,  -5,   3,   1,   1,   3,  -5,   10],
    [ -20, -40,  -5,  -5,  -5,  -5, -40,  -20],
    [ 100, -20,  10,   5,   5,  10, -20,  100],
], dtype=int)

# PLAYER TYPES: "1" Random  "2" Heuristic  "3" Minimax  "4" Minimax AB
#               "5" MCTS    "6" Descent
# CNN variants: "1c" "2c" "3c" "4c" "5c" "6c" (same strategy, CNN eval)
```

Listing 6: Board representation and positional weight map (Block 1) for the Othello engine.

The comment in the last two lines of Listing 6 anticipates the rest of this report: every one of the six search strategies developed in Chapters 7 and 8 is registered twice, once with the handcrafted heuristic and once with the CNN evaluator described in Chapter 10, giving twelve directly comparable AI configurations in total.

### 6.4 Preserving Continuity Across Phases

Although the game, the dataset, and eventually the evaluation function all change in this phase, several conventions are deliberately preserved from Connect 4: in-place move application with explicit undo rather than board copying, a dictionary-based dispatcher (`AI_FUNCTIONS`) mapping strategy identifiers to callable agents, and a clean separation between game logic, evaluation, and interface. This continuity is what makes it possible, later in this report, to treat the switch from a handcrafted evaluator to a CNN evaluator as a controlled, single-variable change rather than a full architectural rewrite — the same discipline that made the learning-rate and optimizer sweeps in Chapter 5 interpretable.

## 7 Search Algorithms in the Othello Engine: Minimax, Alpha-Beta Pruning and MCTS

### 7.1 Overview

The Othello engine implements six search strategies of increasing sophistication, all operating on the same global board state and sharing the same move-generation and disc-flipping primitives. Every strategy is registered in a common `AI_FUNCTIONS` dispatcher, so that any two of them can be pitted against each other without changing the surrounding game loop — the same dispatcher pattern introduced in the Connect 4 phase (Chapter 4) and now extended to a much richer set of agents. This chapter covers the first four strategies together with Monte Carlo Tree Search; the sixth and most recent strategy, Descent Minimax, is developed separately in Chapter 8, since it is grounded in a specific piece of published research.

### 7.2 Random Baseline (Strategy 1)

As in every previous phase, the simplest agent, `PlayAIRandom`, selects a legal move uniformly at random. It requires no evaluation function whatsoever and serves two purposes: as a sanity check on the surrounding game logic (every more sophisticated agent should beat it reliably), and as the fixed lower bound against which every win-rate comparison in Chapter 12 is anchored. Against stronger agents, random play typically loses by disc margins of 30 or more.

### 7.3 Heuristic Greedy (Strategy 2)

The heuristic agent, `PlayAIHeuristic`, selects the move with the highest score according to a fixed priority ladder implemented in `GetScoreForPosition`: a move that immediately wins the game scores 100; a move that blocks an opponent's win scores 50; capturing a corner scores 30; every other move is scored by `GetMoveScore`, which linearly combines a positional weight from `POSITION_MAP` (corners +100, X-squares −40, introduced in Listing 6), the number of discs flipped (weight 3), and a mobility penalty proportional to the number of legal moves left to the opponent.

The separation between `GetScoreForPosition` and `GetMoveScore` is a deliberate performance optimization: `GetScoreForPosition` uses early-exit logic to avoid calling the more expensive flip-counting routine for winning, blocking, or corner moves, while `GetMoveScore` remains a pure, reusable scoring primitive that deeper strategies — including Minimax — also call directly.

The full board-evaluation function used by every deeper strategy, `EvaluateGrid`, combines four weighted components: positional score (weight 3), mobility difference (weight 5), a frontier-disc penalty (weight 2 — a frontier disc being one adjacent to at least one empty cell, and therefore more exposed), and disc-count difference (weight 1, tripled once more than 48 discs are on the board, since raw disc count becomes far more decisive in the endgame).

### 7.4 Minimax Search (Strategy 3)

`Minimax` implements the standard adversarial tree search with no pruning, alternating between maximizing nodes (the AI's turn) and minimizing nodes (the opponent's turn), descending to a fixed depth (4 plies by default) before invoking `EvaluateGrid` at the leaves. A important rule-specific detail is the handling of forced passes: if the player to move has no legal moves, the turn passes to the opponent *without* consuming a depth level, correctly implementing the Othello rule that a player must pass if and only if they have no legal moves, and that the game ends only once both players have passed consecutively.

As in every previous phase, move application avoids copying the board: `FlipDiscs` applies a move in place, and an explicit undo sequence restores the played cell to empty and every flipped disc to the opponent, which is faster than copying the $8 \times 8$ array at every node of the search tree.

### 7.5 Minimax with Alpha-Beta Pruning (Strategy 4)

Strategy 4, `MinimaxAB`, adds alpha-beta pruning on top of the same traversal: `alpha` tracks the best score the maximizer can already guarantee along the current path, `beta` tracks the best score the minimizer can already guarantee, and any subtree is pruned as soon as `alpha >= beta`, since it cannot influence the decision at the root. Combined with move ordering — trying the most promising moves first via `_sort_moves`, so that strong cutoffs happen early — alpha-beta pruning is theoretically optimal when moves are ordered best-first, reducing the effective branching factor from $b$ to roughly $\sqrt{b}$ in the best case, so that the same depth can be searched in $b^{d/2}$ node evaluations instead of $b^d$. Without perfect move ordering the gain is smaller, but it remains substantial, especially at greater search depths. Listing 7 shows the pruning core, including the forced-pass handling inherited from plain Minimax.

```
def MinimaxAB(depth, is_maximizing, p_id, opp_id, alpha, beta, eval_fn):
    state = CheckGameState()
    if depth == 0 or state != 0:
        return eval_fn(p_id, opp_id)

    current_player  = p_id if is_maximizing else opp_id
    opponent_player  = opp_id if is_maximizing else p_id
    possible_moves   = sort_moves(GetPossibleMoves(current_player))

```

```
    if not possible_moves:                          # forced pass
        if not GetPossibleMoves(opponent_player): # both stuck: terminal
            return eval_fn(p_id, opp_id)
        return MinimaxAB(depth, not is_maximizing, p_id, opp_id,
                         alpha, beta, eval_fn)

    if is_maximizing:
        best_val = -float('inf')
        for row, col in possible_moves:
            flipped = GetFlippedDiscs(row, col, current_player)
            FlipDiscs(row, col, current_player, flipped)
            value = MinimaxAB(depth - 1, False, p_id, opp_id, alpha, beta,
    eval_fn)
            Grid[row][col] = 0
            for r, c in flipped: Grid[r][c] = opponent_player
            best_val = max(best_val, value)
            alpha = max(alpha, best_val)
            if alpha >= beta: break                # beta cutoff
        return best_val
    else:
        best_val = float('inf')
        for row, col in possible_moves:
            flipped = GetFlippedDiscs(row, col, current_player)
            FlipDiscs(row, col, current_player, flipped)
            value = MinimaxAB(depth - 1, True, p_id, opp_id, alpha, beta,
    eval_fn)
            Grid[row][col] = 0
            for r, c in flipped: Grid[r][c] = opponent_player
            best_val = min(best_val, value)
            beta = min(beta, best_val)
            if alpha >= beta: break                # alpha cutoff
        return best_val
```

Listing 7: Alpha-beta pruning with move ordering and forced-pass handling for Othello.

### 7.6 Monte Carlo Tree Search (Strategy 5)

Strategy 5 implements a full UCT (Upper Confidence bounds applied to Trees) variant of Monte Carlo Tree Search, running for a fixed time budget of 1.0 second per move rather than to a fixed depth — an echo of the empirical, hardware-calibrated time budgeting first introduced with the Connect 4 "one-second limit" (Section 4). Each iteration consists of four phases:

- **Selection.** Starting from the root, descend via UCB1 selection until reaching a node with untried moves or a terminal node.
- **Expansion.** If the node has untried moves, create a new child for one of them.
- **Simulation (rollout).** From the new node, play a random rollout up to 60 moves deep, returning $+1$ if the AI wins, $-1$ if it loses, 0 for a draw.
- **Backpropagation.** Propagate the result back to the root, incrementing the visit count $N$ and the win sum $W$ of every node on the path.

The exploration–exploitation trade-off is governed by the UCB1 formula, $W/N + C\sqrt{\ln(N_{\text{parent}})/N}$, with exploration constant $C = \sqrt{2}$. Once the time budget expires, the move corresponding to the child with the *highest visit count* is selected — not the highest average reward — since visit count is a more stable estimator at the root than a reward average computed over possibly few simulations. Listing 8 shows the node structure and the main search loop.

```
class MCTSNode:
    __slots__ = ("state", "turn", "parent", "move",
                 "children", "untried", "W", "N")

    def __init__(self, state, turn, parent=None, move=None):
```

```
        self.state, self.turn     = state, turn
        self.parent, self.move    = parent, move
        self.children, self.untried = [], _mcts_legal_moves(state, turn)
        self.W, self.N = 0.0, 0

    def ucb1(self, c):
        if self.N == 0:
            return float('inf')
        return self.W / self.N + c * math.sqrt(math.log(self.parent.N) / self.N)

    def best_child(self, c):
        return max(self.children, key=lambda ch: ch.ucb1(c))


def _mcts_run(root_state, root_turn, p_id, opp_id, use_cnn=False):
    root = MCTSNode(state=root_state, turn=root_turn)
    t0 = time.time()
    while time.time() - t0 < MCTS_TIME_BUDGET:
        leaf = _mcts_select(root)
        if not leaf.is_terminal():
            leaf = _mcts_expand(leaf)
        result = (_mcts_cnn_result(leaf.state, p_id, opp_id) if use_cnn
                  else _mcts_rollout_result(leaf.state, p_id, opp_id))
        _mcts_backpropagate(leaf, result)
    return root
```

Listing 8: MCTS node (UCB1) and main search loop for Othello.

Note the `use_cnn` flag already present in `_mcts_run`: it is not used in this chapter (where every rollout is a genuine random playout), but it is the exact hook that Chapter 10 uses to replace the rollout step with a single CNN forward pass. The MCTS tree itself is built from lightweight board *snapshots* stored in each `MCTSNode`, decoupled from the shared global `Grid`. This is a necessary departure from the in-place, undo-based approach used by Minimax and the heuristic agent: MCTS explores many branches of the tree concurrently (as nodes accumulate statistics across iterations), which is incompatible with a single mutable global board.

# 8 The Descent Algorithm: Cohen-Solal's Contribution

## 8.1 Minimax Strikes Back

The sixth and most recent search strategy implemented in the Othello engine, Descent Minimax, is not an original invention of this project but a direct implementation of an algorithm proposed by Cohen-Solal and Cazenave in *Minimax Strikes Back* [1], published at AAMAS 2023. Understanding why this algorithm is included, and what makes it different from the five strategies already covered, requires briefly summarizing the paper's argument.

Since AlphaGo and AlphaZero [33, 25], the dominant paradigm for learning to play board games without prior knowledge has been Monte Carlo Tree Search guided by a neural network: a policy network biases exploration toward promising moves, and a value network replaces the noisy random rollouts that classical MCTS relied on. Cohen-Solal and Cazenave's central claim is that this is not the only — nor necessarily the best — way to combine search with reinforcement learning. They propose **Athénan**, a learning framework built instead around a Minimax-based search algorithm, and show that it is competitive with, and in several games superior to, Polygames (an open-source AlphaZero reimplementation), while requiring far less computation: the paper reports the cost of generating training data as roughly 296 times lower with Athénan, and, using comparable resources, Athénan is at least 7 times faster than Polygames without a reinforcement-learned heuristic and more than 30 times faster with one — remaining competitive even when Polygames is given 100 times more GPU resources.

## 8.2 Descent: Extending the Search to Terminal States

The Athénan framework actually relies on two related but distinct algorithms, used at two different moments:

- During **learning**, Athénan uses **Descent Minimax** (or simply *Descent*): at each iteration, it extends the current best sequence of actions — the principal variation of the partial game tree built so far — one step further, all the way to a terminal state, rather than stopping at an arbitrary leaf as classical Minimax or MCTS do.
- During **confrontations** (i.e. actual game play against an opponent), Athénan instead uses a variant called **Unbounded Best-First Minimax with Safe Decision**, which performs the same kind of iterative tree extension but selects the move to play based on the most-explored action rather than the highest-valued one, trading some short-term optimality for more robust decisions.

The key property that motivates using Descent specifically during learning is that it does not lose information. Because each iteration explores all the way to a terminal state, every value computed during the search is a legitimate training target — there is no need to run separate, parallel self-play matches purely to generate a training corpus, as AlphaZero-style approaches do. This is also the reason the paper's efficiency claims (far cheaper data generation, no need for match parallelization) hold: the search process itself *is* the data-generation process.

For the purposes of this project, the relevant contribution is not the reinforcement-learning framework around Descent (Athénan trains its own value network end-to-end; this project instead pairs Descent with the two evaluation functions already used by every other strategy: the handcrafted `EvaluateGrid` in this chapter, and the trained CNN evaluator in Chapter 10) but the search algorithm itself: a persistent, incrementally-refined tree search that keeps improving its evaluation of promising lines without repeatedly re-exploring the branches it has already ruled out.

## 8.3 Implementation: Descent Minimax in the Othello Engine

Strategy 6, `PlayAIDescent`, adapts Descent to the time-budgeted, single-move-at-a-time setting of the rest of the engine (rather than the full offline learning loop described in the paper). It maintains a persistent dictionary `S`, mapping board states — keyed by a flat tuple of the grid contents plus the player to move, via `_board_key` — to their current heuristic evaluation and, for non-terminal states, a table of edge values indexed by move.

Each call to `descent_iter` performs one iteration: it looks up the current state's key in `S`; if the state is new, it expands every legal move once, evaluating each resulting child position and recording it as an edge value. It then follows the *best known move so far* (the greedy path through the edge table — the local equivalent of the principal variation), recurses one level deeper along that path, and, on returning, updates the edge value of the move it just took with the freshly computed result. Over many iterations within the same move decision, this progressively sharpens the evaluation of the subtree reachable from the current position, concentrating computation on the lines that already look most promising rather than spreading it uniformly as plain Minimax does. Listing 9 shows the complete implementation.

```
def descent_iter(turn, S, p_id, opp_id, eval_fn):
    key   = _board_key(turn)
    state = CheckGameState()

    if state != 0:                                    # terminal: evaluate and
    store
        val    = _heuristic_eval(p_id, opp_id, eval_fn)
        S[key] = {"value": val, "turn": turn}
        return val

```

```
    if key not in S:                                    # first visit: expand once
        next_turn      = opp_id if turn == p_id else p_id
        possible_moves = GetPossibleMoves_fast(turn)
        if not possible_moves:
            return descent_iter(next_turn, S, p_id, opp_id, eval_fn)

        S[key] = {"value": None, "turn": turn, "edge": {}}
        for move in possible_moves:
            flipped = _apply_move(move, turn)
            if flipped is None:
                continue
            child_state = CheckGameState()
            if child_state != 0:
                S[key]["edge"][move] = _heuristic_eval(p_id, opp_id, eval_fn)
            else:
                S[key]["edge"][move] = _heuristic_eval(p_id, opp_id, eval_fn)
            _undo_move(move, turn, flipped)

    edges = S[key]["edge"]
    if not edges:
        return _heuristic_eval(p_id, opp_id, eval_fn)

    # Follow the currently best-known move (the greedy / principal-variation path)
    best_move = (max if turn == p_id else min)(edges, key=lambda m: edges[m])
    next_turn = opp_id if turn == p_id else p_id
    flipped   = _apply_move(best_move, turn)
    if flipped is not None:
        updated_val = descent_iter(next_turn, S, p_id, opp_id, eval_fn)  # descend one level
        _undo_move(best_move, turn, flipped)
        S[key]["edge"][best_move] = updated_val                          # refine on return
        best_move = (max if turn == p_id else min)(edges, key=lambda m: edges[m])

    S[key]["value"] = edges[best_move]
    return S[key]["value"]


def descent(start_turn, S, p_id, opp_id, tau, eval_fn):
    """Outer, time-bounded loop: repeatedly refine the tree for tau seconds."""
    t0 = time.time()
    while time.time() - t0 < tau:
        descent_iter(start_turn, S, p_id, opp_id, eval_fn)
    return S


def PlayAIDescent(player_id, tau=1.0, use_cnn=False):
    opp_id  = 3 - player_id
    eval_fn = _get_eval_fn(use_cnn)
    S       = {}
    descent(player_id, S, player_id, opp_id, tau, eval_fn)
    # ... best move at the root is then read from S and applied to Grid
```

Listing 9: Descent Minimax: persistent tree, greedy descent, and time-bounded outer loop.

Two properties of this implementation are worth highlighting. First, unlike Minimax and Alpha-Beta, which rebuild their search tree from scratch at every move, Descent's dictionary `S` persists *within* a single move decision across many calls to `descent_iter`, so repeated iterations improve the evaluation of promising moves without re-evaluating branches that earlier iterations already explored and rejected. Second, the algorithm naturally balances exploration

and exploitation: because it always descends along the currently best-known move, it concentrates search effort where it is most likely to matter, while the continual re-evaluation on the way back up keeps that "best-known move" honest as deeper information arrives. This combination — persistent, incrementally-refined search rather than a fixed-depth tree rebuilt from nothing — is precisely what makes Descent theoretically well suited to the regime where the time budget is small relative to the branching factor, the same regime that motivates using MCTS in the first place, which is why Chapter 12 compares the two head-to-head.

### 8.4 A Note on Chronology

For completeness, it is worth noting that a working prototype of Descent Minimax (`descent_iter`, referencing Cohen-Solal and Cazenave directly in its comments) already existed in the Connect 4 code base described in Chapter 4, registered as strategy "5" alongside Random, Heuristic, and Minimax. It was not, however, documented in the Connect 4 report, and Connect 4's fully solved, comparatively small search space makes it a far less interesting setting for evaluating Descent than Othello. The version described in this chapter, integrated with the CNN evaluator (Chapter 10) and benchmarked statistically (Chapter 12), is the one this report treats as the algorithm's proper introduction into the project.

## 9 Recap: Comparative Overview of the Six Implemented Strategies

Chapters 7 and 8 introduced six search strategies of increasing sophistication, each registered under the same `AI_FUNCTIONS` dispatcher and sharing the same move-generation, disc-flipping, and (where applicable) evaluation primitives. Before turning to the CNN evaluator in Chapter 10, it is useful to lay all six side by side: Table 4 summarizes how each one decides which move to play, how it stops searching, and whether it keeps any memory of previous decisions.

| Strategy | Search paradigm | Stopping criterion | Tree memory | Key idea |
|---|---|---|---|---|
| 1. Random | None | Immediate | None | Uniform random legal move; sanity check and win-rate floor. |
| 2. Heuristic | Single-ply greedy | Immediate | None | Priority ladder (win > block > corner > weighted score). |
| 3. Minimax | Full-width tree search | Fixed depth (4 plies) | Rebuilt every move | Exhaustive adversarial search to a fixed horizon, `EvaluateGrid` at the leaves. |
| 4. Minimax + $\alpha\beta$ | Pruned tree search | Fixed depth, pruned | Rebuilt every move | Same as Minimax, but prunes subtrees once $\alpha \geq \beta$; move ordering improves cutoffs. |
| 5. MCTS (UCT) | Best-first, sampling-based | Fixed time budget (1.0 s) | Rebuilt every move (root re-initialized) | Selection / expansion / rollout / backpropagation, guided by UCB1. |
| 6. Descent | Best-first, incremental | Fixed time budget | **Persistent** across iterations within a move | Repeatedly extends the greedy (principal-variation) path to a leaf, refining edge values on the way back up. |

Table 4: Comparative summary of the six search strategies implemented in the Othello engine.

A few groupings emerge from this comparison. Strategies 1 and 2 require no search tree at all and no leaf evaluation function beyond a single-move scoring heuristic; strategies 3 and 4 are exact, fixed-depth tree searches that differ only in whether unproductive branches are pruned; and strategies 5 and 6 both operate under a wall-clock time budget rather than a fixed depth, but differ fundamentally in how they use that time — MCTS spends it sampling many random rollouts across a broadly explored tree, while Descent spends it repeatedly deepening a single, continuously-refined best-known line.

This last distinction matters for the next chapter. Every strategy from 3 onward relies on a leaf (or rollout) evaluation to turn a non-terminal board into a number: Minimax, Alpha-Beta, and Descent call `EvaluateGrid` directly at their leaves, while MCTS instead estimates a leaf's value indirectly, by averaging the outcome of many random rollouts played out from it. Chapter 10 shows that this evaluation step — whichever form it takes — can be replaced by a single trained CNN forward pass without touching any of the surrounding search logic summarized in Table 4: a direct substitution of `EvaluateGrid` by `EvaluateGridCNN` for Minimax, Alpha-Beta, and Descent, and a substitution of the random rollout itself for MCTS.

# 10 Integrating the CNN Evaluator: OthelloCNN and Neural-Guided Search

This chapter closes the loop opened in Chapter 6: having motivated the switch to Othello by the availability of the WTHOR dataset, and having laid out six search strategies built around a handcrafted evaluation function (Chapters 7–9), it now describes the network trained to replace that handcrafted function, and the mechanics of plugging it into every one of the six strategies.

## 10.1 Design Rationale: Why Not TinyCNN Sport?

It is tempting to assume that the CNN trained in Chapter 5 could simply be repurposed here. It cannot, for two structural reasons. First, TinyCNN Sport uses three max-pooling stages to reduce a $64 \times 64$ input down to a class label; applied to an $8 \times 8$ Othello board, a single max-pooling step would already collapse it to $4 \times 4$, destroying half the spatial information in one operation. Since the positional identity of each cell — corner, edge, or center — is central to Othello strategy, preserving spatial resolution throughout the network is non-negotiable, ruling out pooling entirely. Second, the output type is different: TinyCNN Sport predicts a categorical label via a softmax head, whereas the Othello network must predict a continuous value (the final disc margin, in $[-64, +64]$), so the output head is a single linear unit with no activation, trained with Mean Squared Error rather than cross-entropy loss.

## 10.2 Architecture: OthelloCNN

OthelloCNN follows a residual architecture [15] with three components, summarized in Table 5: a stem that maps the 3-channel input to 64 feature maps, a body of four residual blocks that deepen the representation while preserving the full $8 \times 8$ spatial resolution throughout, and a head that collapses the spatial dimensions to a single scalar margin prediction. The network has 300,481 trainable parameters, almost all of them in the eight $64 \to 64$ convolutions of the residual body — small enough to run thousands of inferences per second on a CPU, which is essential given that Section 10.7 calls it at every leaf node of a live tree search.

| Component | Specification | Purpose |
|---|---|---|
| Stem | `Conv2d(3,64,3,p=1)`, BN, ReLU | Maps 3 input channels to 64 feature maps ($3 \times 3$ receptive field per cell) |
| Body ($\times 4$) | ResBlock(64): two `Conv2d(64,64,3,p=1)` + BN, skip connection | Deepens representations without reducing spatial size; skip connections prevent gradient vanishing |
| Head | `AdaptiveAvgPool2d(1,1)`, Flatten, `Linear(64,32)`, ReLU, `Linear(32,1)` | Collapses spatial dimensions to a scalar margin prediction |

Table 5: OthelloCNN architecture (300,481 trainable parameters).

Every residual block preserves the $8 \times 8$ resolution throughout the body via $3 \times 3$, padding-1 convolutions; the skip connection, $\text{out} = \text{ReLU}(\text{conv2}(\text{BN}(\text{conv1}(x))) + x)$, keeps gradients flowing to the early layers even through four stacked blocks, letting the network learn both local cell-level patterns and composite patterns such as corner control, disc stability, and mobility influence. Listing 10 shows the full architecture, exactly as defined for both training and inference.

```
class ResBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn1   = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn2   = nn.BatchNorm2d(channels)

    def forward(self, x):
        residual = x
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + residual)
```

```
14
15
16 class OthelloCNN(nn.Module):
17     def __init__(self, channels=64, n_blocks=4):
18         super().__init__()
19         self.stem = nn.Sequential(
20             nn.Conv2d(3, channels, kernel_size=3, padding=1),
21             nn.BatchNorm2d(channels),
22             nn.ReLU(inplace=True),
23         )
24         self.body = nn.Sequential(*[ResBlock(channels) for _ in range(n_blocks)
    ])
25         self.head = nn.Sequential(
26             nn.AdaptiveAvgPool2d((1, 1)),
27             nn.Flatten(),
28             nn.Linear(channels, 32),
29             nn.ReLU(inplace=True),
30             nn.Linear(32, 1),
31         )
32
33     def forward(self, x):
34         x = self.stem(x)
35         x = self.body(x)
36         x = self.head(x)
37         return x.squeeze(1)      # predicted disc margin, Black's perspective
```

Listing 10: OthelloCNN: residual architecture for board evaluation by regression.

### 10.3 Board Encoding

Each board position is encoded as a $(3, 8, 8)$ float32 tensor: channel 0 marks cells occupied by Black (1.0 where present, 0.0 elsewhere), channel 1 marks cells occupied by White, and channel 2 is a turn plane — entirely 1.0 if it is Black's turn to move, entirely 0.0 otherwise. The turn plane is not optional: the same physical arrangement of discs has a different strategic value depending on who moves next, and without this channel the network would have no way to distinguish the two cases. The network is always trained to predict the margin (Black discs minus White discs at game end) from Black's perspective; calling code flips the sign of the prediction whenever the evaluation is requested from White's point of view.

### 10.4 Dataset and Augmentation

Training data is derived from the same WTHOR files described in Section 6: every game is replayed move by move, positions before move 10 are discarded (the stereotyped opening carries little discriminating signal), and every remaining position is paired with that game's final disc margin as its regression target. During training, the `OthelloDataset` class applies random dihedral symmetry augmentation: one of Othello's 8 symmetries (4 rotations $\times$ 2 mirror reflections) is sampled uniformly and applied to the board tensor at load time. Since the disc margin is invariant under every one of these transformations, this multiplies the effective dataset size by up to 8 at no additional labelling cost — reusing, in a new domain, exactly the kind of augmentation-as-regularizer strategy that proved so effective for TinyCNN Sport in Chapter 5.

### 10.5 Training Protocol

OthelloCNN is trained with SGD (momentum 0.9, weight decay $10^{-4}$) under a multi-step learning-rate schedule dropping by a factor of 10 at epochs 30 and 50 (from an initial 0.01 down to 0.001 then 0.0001), for 60 epochs at batch size 256, on Google Colab with GPU acceleration. The training script mirrors the structure used for TinyCNN Sport: a `createScenario`

function assembles all training components into a scenario object, and a `train` function runs the epoch loop. The training script (`05_othello_cnn.py`) and its full per-epoch log are available in the thesis repository [37].

**Game-level split.** The 80/20 train/validation split is performed at the level of *games*, not of positions: the WTHOR games are shuffled with a fixed seed and divided before being expanded into positions, so that all the positions of a given game fall on the same side of the split. This matters because consecutive positions of the same game share most of their discs and carry the same target; a split performed on the flat pool of positions would place near-duplicates on both sides and make the validation metrics optimistic. The resulting split contains 67,929 training games (3,387,973 positions) and 16,983 validation games (847,308 positions). All random generators involved – Python, NumPy and PyTorch, including the data-loading workers that apply the augmentation – are seeded, so the split, the weight initialisation, the shuffling order and the augmentation are all reproducible.

**Gradient clipping.** At every step, the global $L_2$ norm of the gradient is clipped to 1.0. The targets are unnormalised disc margins in $[-64, +64]$, so the MSE gradients of the first training steps can be very large; earlier training attempts without clipping occasionally collapsed to a constant output, a single oversized update having driven all the ReLU units of the 32-unit head into their inactive region (the "dead ReLU" problem), after which no gradient can reach them again. Clipping bounds the size of each update and removes this failure mode. To detect a collapse as soon as it occurs, the standard deviation of the validation predictions is logged at every epoch: it would tend to zero for a constant output, and in the run reported here it remained between 9.4 and 10.7 discs throughout training.

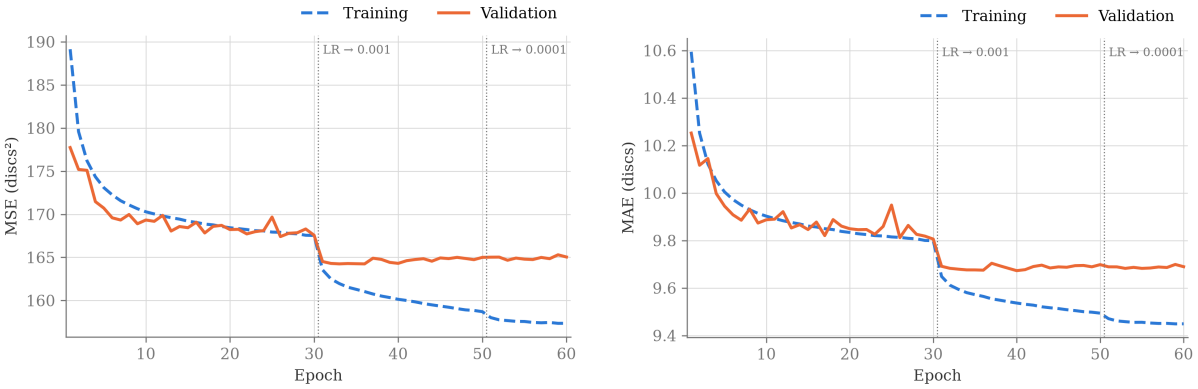
**Logged metrics.** At every epoch the script logs the learning rate, the MSE loss and mean absolute error (MAE) on both sets, and four metrics computed on the full validation set: the Pearson correlation $r$ and the Spearman rank correlation $\rho$ between predicted and true margin, the coefficient of determination $R^2$, and the *sign accuracy*, i.e. the share of decisive positions (true margin $\neq 0$) whose predicted sign matches the actual winner. The log and the model weights are saved to Google Drive every 5 epochs; after the last epoch, the prediction for every validation position is exported together with the number of moves already played, for the analysis below. The model used throughout the rest of this thesis is the final one, at epoch 60.

## 10.6 Training Results

Table 6 reports the metrics of the final model, and Figures 6–8 their evolution during training and their distribution over the validation set.

| **Metric** | **Training** | **Validation** |
|---|---:|---:|
| MSE (discs$^2$) | 157.36 | 165.04 |
| MAE (discs) | 9.45 | 9.69 |
| Pearson $r$ | – | 0.618 |
| Spearman $\rho$ | – | 0.573 |
| $R^2$ | – | 0.380 |
| Sign accuracy | – | 73.3% |
| MSE of a constant predictor (target variance) | – | 266.3 |

Table 6: OthelloCNN metrics at the end of training (epoch 60). The last row is the MSE of a predictor that always outputs the mean margin, given for reference.

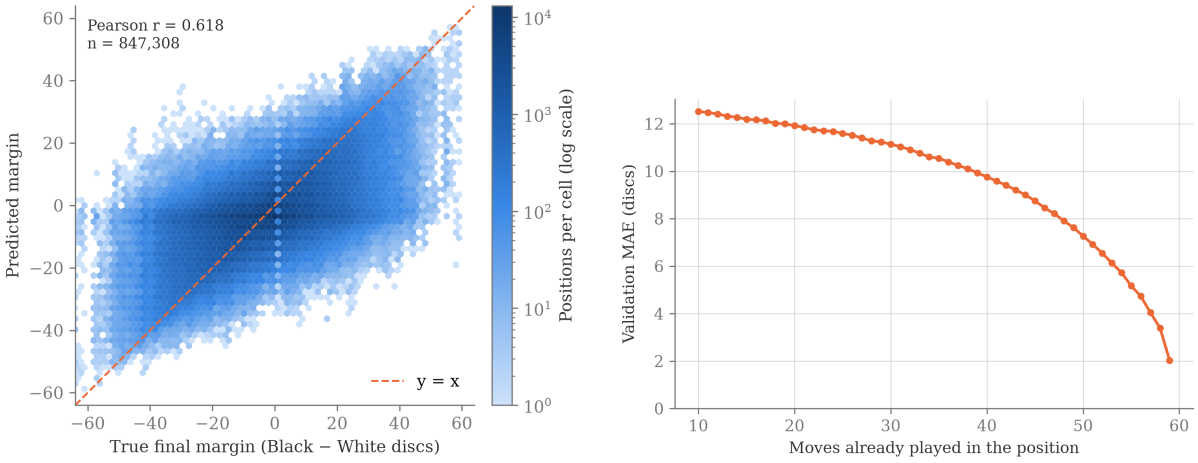



(a) MSE loss

(b) Mean absolute error

Figure 6: OthelloCNN training (dashed) and validation (solid) curves over 60 epochs. Dotted vertical lines mark the learning-rate drops.

**Learning dynamics.** During the first 30 epochs, at a learning rate of 0.01, training and validation losses decrease together and stay within about one unit of each other (validation MSE from 177.7 at epoch 1 to 167.6 at epoch 30), with the epoch-to-epoch fluctuations of the validation loss typical of a relatively high learning rate. The first learning-rate drop produces an immediate step down in both curves (validation MSE 164.5 at epoch 31). From then on the two curves diverge: the training loss keeps decreasing, down to 157.4 at epoch 60, while the validation loss stays flat, reaching its minimum of 164.24 at epoch 36 and ending at 165.04. This is a mild form of overfitting in the last 25 epochs, small in magnitude – the final validation MSE is 0.5% above its minimum – and the second learning-rate drop at epoch 51 has no visible effect on the validation curve. Stopping at epoch 36 would therefore not have changed the model materially, and the final model is retained for consistency with the protocol. Figure 8 shows that the correlation metrics follow the same pattern: most of the gain is obtained in the first few epochs (Pearson $r$ from 0.578 at epoch 1 to 0.610 at epoch 30), the first learning-rate drop adds a further small step (to 0.618), and all four metrics then remain stable.




(a) Predicted vs true margin

(b) Error along the game

Figure 7: Final model on the 847,308 validation positions: (a) density of predicted against true final margin, with the $y = x$ line; (b) validation MAE as a function of the number of moves already played.

**Interpretation.** With $R^2 = 0.380$, the network explains 38% of the variance of the final margin over unseen games, reducing the MSE of a constant predictor from 266.3 to 165.0. This moderate level reflects the nature of the target rather than a deficiency of the model: every position from move 10 onward is labelled with the final margin of a human game, which also depends on all the moves still to be played, so a large part of the target is unpredictable from the position alone. Figure 7(b) makes this explicit: the validation MAE is about 12.5 discs for positions after 10 moves, decreases steadily through the middle game, falls faster in the endgame as the outcome becomes determined, and is about 2 discs one move before the end. Figure 7(a) shows the corresponding behaviour over the whole validation set: the predictions follow the diagonal but are compressed towards zero, as expected from a regressor trained with MSE on a noisy target, which learns the conditional mean of the outcome. The spread of the predictions (standard deviation 10.6 discs) is consistent with that of the targets scaled by the correlation ($0.618 \times 16.3 \approx 10.1$ discs), indicating that the network is not over-confident. For its use inside a search, what matters most is that the network ranks positions correctly; the sign accuracy of 73.3% means that, already from move 10 onward, the predicted leader is the actual winner in almost three positions out of four. Because the split is performed at game level, all these figures are honest estimates of the network's behaviour on games it has never seen.

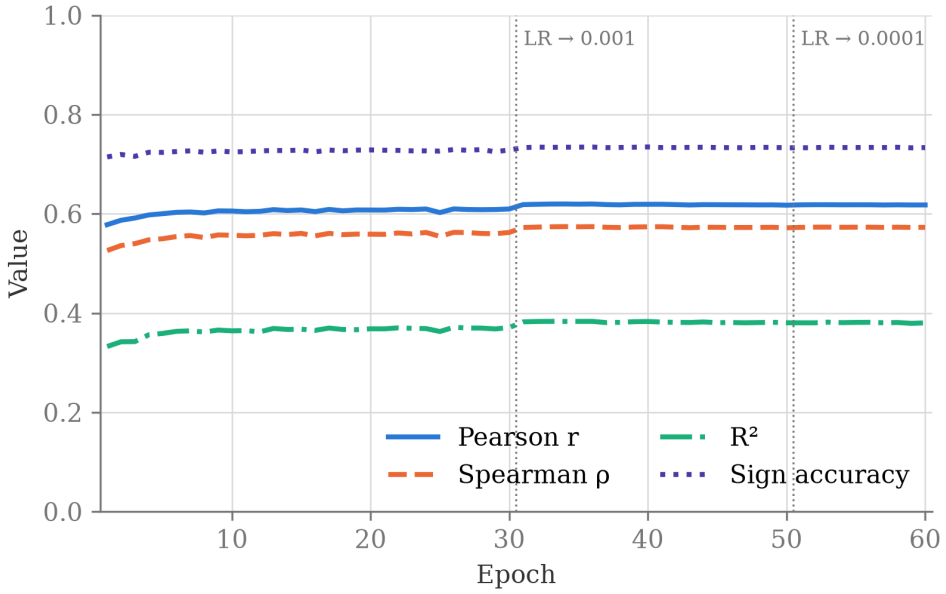



Figure 8: Validation correlation metrics over 60 epochs: Pearson $r$, Spearman $\rho$, $R^2$ and sign accuracy.

## 10.7 CNN Integration into Search Algorithms

### 10.7.1 Design Principle: Drop-In Replacement

The integration principle governing this entire section is that `EvaluateGridCNN` is a strict drop-in replacement for the handcrafted `EvaluateGrid`: both functions accept the same arguments (`p_id`, `opp_id`) and return a signed float representing the advantage for `p_id`, so every search strategy can select its evaluation function at construction time through a single dispatcher, `_get_eval_fn(use_cnn)`, without changing a single line of search logic. This yields twelve distinct AI configurations in total: the six base strategies of Chapters 7–8, each available with either the handcrafted heuristic (dispatcher keys `"1"`–`"6"`) or the CNN evaluator (keys `"1c"`–`"6c"`), with automatic fallback to the heuristic if the trained model file is missing.

### 10.7.2 EvaluateGridCNN: Implementation and Sign Convention

`EvaluateGridCNN` reads the global board, encodes it as a $(1, 3, 8, 8)$ tensor, and runs it through a lazily-loaded, module-cached model in `torch.no_grad()` mode; only the first call pays the cost of reading weights from disk. Terminal states bypass the network entirely and are scored by the exact `EvaluateGrid` ($\pm 500$), both because the network was never trained on end-of-game positions and to guarantee correct terminal behaviour regardless of model quality. If PyTorch is unavailable or the weights file is missing, the function transparently falls back to the heuristic, so the game remains playable in every environment. Listing 11 shows the complete implementation, including the sign convention that keeps it consistent with `EvaluateGrid`.

```
def EvaluateGridCNN(p_id, opp_id):
    state = CheckGameState()
    if state != 0:                          # terminal: exact heuristic, no network
     call
        return EvaluateGrid(p_id, opp_id)
    if not _cnn_available:
        return EvaluateGrid(p_id, opp_id)  # graceful fallback

    with torch.no_grad():
        x      = _board_to_tensor_local(p_id).to(_cnn_device)
        margin = _cnn_model(x).item()    # predicted Black - White margin

    # Flip sign for White so positive always means "p_id is winning"
    return margin if p_id == 1 else -margin


def _get_eval_fn(use_cnn):
    if use_cnn and LoadCNN():
        return EvaluateGridCNN
    return EvaluateGrid
```

Listing 11: `EvaluateGridCNN`: drop-in neural replacement for the handcrafted evaluator.

### 10.7.3 Minimax and Alpha-Beta + CNN (Strategies 3c, 4c)

For both plain Minimax and Minimax with Alpha-Beta pruning, the CNN simply replaces the leaf-evaluation call; search tree structure, depth limit, and move enumeration are otherwise identical to the heuristic variants (Chapter 7). The expected benefit is that the network may capture positional correlations the handcrafted, linearly-weighted `EvaluateGrid` formula misses. The cost is inference latency: a single CNN forward pass takes roughly 0.2–2 ms on CPU, against microseconds for the heuristic; at depth 4 with a branching factor around 8–10, this can mean several thousand network calls per move, which fits comfortably inside a 1-second budget but constrains how deep the CNN-guided search can practically go compared to its heuristic counterpart.

### 10.7.4 MCTS + CNN (Strategy 5c): Replacing the Rollout

The integration with MCTS is qualitatively different from Minimax: rather than replacing a leaf evaluation after a fixed-depth descent, the CNN replaces the *random rollout itself* — directly mirroring the design choice that distinguishes AlphaZero-style MCTS from classical MCTS, where the noisy, expensive random playout is replaced by a single neural network judgement. Listing 12 shows `_mcts_cnn_result`: it temporarily writes the leaf's board snapshot into the global grid, calls `EvaluateGridCNN`, restores the original board, and normalizes the result to $[-1, +1]$ by dividing by 64 (the maximum possible disc margin) so it can be backpropagated through the tree with exactly the same bookkeeping as a genuine rollout result.

```
def _mcts_cnn_result(state, p_id, opp_id):
    global Grid
    original = Grid.copy()
    Grid[:] = state
    score = EvaluateGridCNN(p_id, opp_id)
    Grid[:] = original
    return float(np.clip(score / 64.0, -1.0, 1.0))   # normalise to [-1, +1]
```

Listing 12: Replacing the MCTS random rollout with a CNN evaluation.

The key advantage is variance reduction: a random rollout can be swung entirely by one bad random move deep in the playout, whereas a CNN evaluation is deterministic and, given a reasonably well-trained model, tracks the true value of the position more closely. Because each CNN call is also cheaper than a full 60-move random playout, the MCTS+CNN agent can complete more iterations within the same time budget while working with lower-variance value estimates at every one of them — a double advantage that Chapter 12 confirms empirically.

### 10.7.5 Descent Minimax + CNN (Strategy 6c)

For Descent Minimax, the CNN replaces the evaluation function called at every edge initialization and refinement step in `descent_iter` (Chapter 8): the persistent state dictionary `S` stores CNN-derived values instead of heuristic ones, and every iterative refinement reuses the same network for consistency. The injection point is exactly the `eval_fn` parameter already threaded through the full call chain from `PlayAIDescent` down to `descent_iter` in Listing 9 — no structural change to the algorithm is required. This combination is of particular theoretical interest: Descent's repeated, incremental refinement can improve on a single CNN evaluation the way it improves on a single heuristic evaluation, while the CNN in turn gives Descent a stronger initial estimate than the handcrafted heuristic to refine from.

## 11 Test Methodology

Twelve AI configurations (six search strategies, each with two possible evaluation functions) is too large a space to compare informally. This chapter describes the automated benchmark harness built to compare them rigorously: how many games are played, how match-ups are chosen, how first-move advantage is controlled for, and how statistical confidence is quantified.

### 11.1 Goal and Design Principles

The benchmark pits selected pairs of strategies against each other in a series of numbered *phases*, each phase corresponding to a single, fixed match-up (e.g. "MCTS vs. Heuristic"). Three principles govern every phase:

- **Statistical rigor over anecdote.** Rather than reporting a handful of illustrative games, every phase reports a two-sided 95% Wilson confidence interval [32] on Player 1's win rate, so that claims of one strategy being "stronger" than another are backed by a quantified level of confidence rather than a small, potentially unrepresentative sample.
- **Fairness with respect to colour.** Because moving first (Black) can itself confer an advantage, every phase splits its games into two equal blocks: Player 1 plays the first half as Black and the second half as White, so that first-move advantage is cancelled out of the aggregate win rate rather than confounded with strategy strength.
- **A hardware-aware, adaptive time budget.** Rather than fixing the number of games in advance, each phase sizes its own sample against a 20-minute target runtime: slow match-ups (e.g. two strategies with expensive search) automatically run fewer games than fast ones (e.g. Random vs. Random). This 20-minute figure is a sizing target, not a hard

ceiling: every phase is additionally guaranteed a minimum of 100 games, and for the slowest match-ups this floor takes precedence over the target, so the phase's actual duration exceeds 20 minutes (Chapter 12 reports these durations directly).

## 11.2 Per-Phase Protocol

Each phase follows the same four-step protocol:

1. **Calibration.** A single sample game between the two configured strategies is played and timed, giving an estimate of the average time per game for this specific match-up.
2. **Sample size selection.** Given that estimate, the harness computes the largest even number of games $n \in [100, 400]$ that fits inside the 20-minute target, rounding down to preserve an exactly symmetric colour split; if even the 100-game floor would exceed that target, the floor is used anyway, guaranteeing every phase a minimum usable sample regardless of how slow the match-up is. In practice this floor governs every phase involving MCTS or Descent, as well as Phase 9 (Table 7), whose per-game cost pushes the 100-game floor's total runtime well past 20 minutes.
3. **Balanced execution.** The $n$ games are run in two blocks of $n/2$: Block A has Player 1 playing Black (moving first) for all $n/2$ games, and Block B has Player 1 playing White for the remaining $n/2$, so that the aggregated result is balanced by construction rather than by chance.
4. **Statistical reporting.** The aggregate win/draw/loss counts and a 95% Wilson confidence interval on Player 1's win rate are computed and logged, together with a plain-language verdict (see below).

Listing 13 shows the calibration and adaptive sample-size logic; Listing 14 shows the statistical core.

```
def _time_one_game(p1, p2):
    """Run a single background game and return elapsed seconds (calibration)."""
    t0 = time.time()
    RunBackgroundGame(p1, p2)
    return time.time() - t0


def _choose_n(avg_sec_per_game, budget_min=20, n_min=100, n_max=400):
    """
    Decide how many games to run given the per-game timing constraint.
      - If even n_min games exceed the budget, run n_min anyway (mandatory floor
    ).
      - Otherwise run as many as possible up to n_max within the budget.
      - n must be even to guarantee a symmetric Black/White colour split.
    """
    budget_sec = budget_min * 60.0
    if avg_sec_per_game > 0:
        n_feasible = int(budget_sec / avg_sec_per_game)
        n_feasible = (n_feasible // 2) * 2       # round down to nearest even
    else:
        n_feasible = n_max
    n = max(n_min, min(n_max, n_feasible))
    return n, avg_sec_per_game * n
```

Listing 13: Speed calibration and adaptive sample-size selection under a fixed time budget.

```
def _wilson_ci(wins, n, z=1.96):
    """Wilson score confidence interval for a proportion (default: 95%)."""
    if n == 0:
        return 0.0, 0.0, 0.0
    p_hat  = wins / n
    denom  = 1 + z**2 / n
    centre = (p_hat + z**2 / (2 * n)) / denom
```

```
    spread = (z * math.sqrt(p_hat * (1 - p_hat) / n + z**2 / (4 * n**2))) /
    denom
    return p_hat, max(0.0, centre - spread), min(1.0, centre + spread)


def _significance_label(ci_low, ci_high):
    """Plain-language verdict from the Wilson CI versus the 50% no-advantage
    line."""
    if ci_low > 0.50:
        return "P1 SIGNIFICANTLY BETTER (p<0.05)"
    if ci_high < 0.50:
        return "P2 SIGNIFICANTLY BETTER (p<0.05)"
    return "No significant difference (CI crosses 50%)"
```

Listing 14: Wilson score confidence interval and significance verdict.

The Wilson interval is preferred here over the simpler normal (Wald) approximation because it remains well-behaved at the sample sizes and win rates involved in this benchmark, including the near-0% and near-100% win rates that several of the stronger match-ups in Chapter 12 produce, where the Wald interval is known to misbehave.

One methodological choice is worth making explicit here: Player 1's "win rate" $\hat{p} = \text{wins}/n$ is computed with $n$ equal to the *total* number of games in the phase, so a draw counts as a non-win for Player 1 in both $\hat{p}$ and its confidence interval, exactly as the `wins` argument in Listing 14 is passed. This is the natural convention when the underlying question is "did Player 1 win or not," and it is the one used throughout Chapter 12. For completeness, the alternative convention – computing the win rate over decisive games only, $\hat{p}_{\text{dec}} = \text{wins}/(\text{wins} + \text{losses})$, with draws removed from the denominator – was also computed post hoc from the per-game logs, using the same Wilson interval. Since draws are rare across the suite (at most 6.0% of games in any single phase, and none at all in five of the ten phases), the two conventions are expected to differ only marginally; Section 12.2 reports the comparison. The harness is implemented in `06_othello.py` [37].

### 11.3 Match-up Design

The full suite consists of ten phases (numbered 1 through 6 and 8 through 11; phase 7 is reserved but not used in the reported comparison), chosen to answer two separate questions in sequence. Phases 1–6 establish the relative strength of the six search strategies under the shared handcrafted evaluator: a sanity check (Random vs. Random), then each successive strategy against a fixed weaker reference (Heuristic vs. Random, Minimax-AB vs. Heuristic, MCTS vs. Heuristic, Descent vs. Heuristic), and finally a direct confrontation between the two strongest search-based strategies (Descent vs. Minimax-AB). Phases 8–11 then isolate the effect of the CNN evaluator alone, holding the search algorithm fixed and only swapping the evaluation function (Heuristic-CNN vs. Heuristic, Minimax-AB-CNN vs. Minimax-AB, MCTS-CNN vs. MCTS, Descent-CNN vs. Descent) — precisely the drop-in substitution described in Chapter 10, now tested empirically rather than only argued for on design grounds.

Every phase is run headlessly (no graphical interface) via `RunBackgroundGame`, which plays a complete game to a terminal state or to two consecutive passes, and `StartSimulation`/`_run_phase`, which repeats this the required number of times and aggregates the outcome. The complete suite was run in two sessions with the same harness and settings: Phases 1–6 in the original run, and Phases 8–11 again after OthelloCNN was retrained with the final protocol of Chapter 10, for a total of about 440 minutes (7.3 hours) of measured play. Individual phase durations range from a few seconds (Random vs. Random) to over 100 minutes (the slowest MCTS and Descent match-ups), a direct empirical illustration of the multiple-orders-of-magnitude cost difference between the six search strategies that Chapter 12 quantifies in full.

### 11.4 Post-hoc Analysis of Colour Effects

The balanced two-block design removes first-move advantage from each phase's *aggregate* win rate by construction, but it does not show whether Player 1's performance depends on the colour it plays within a phase. Since every entry of the per-game log records Player 1's colour, this question can be answered post hoc, without playing any additional game, with the two models described below. Consistently with the main convention, a draw counts as a non-win for Player 1.

**Model 1: per-phase test.** For each phase, the games are arranged in a $2 \times 2$ contingency table:

| | **P1 wins** | **P1 does not win** |
|---|---|---|
| P1 plays Black (Block A) | $a$ | $b$ |
| P1 plays White (Block B) | $c$ | $d$ |

Independence between colour and outcome is tested with Fisher's exact test [34], which is exact for any sample size and remains valid when a cell is empty, as happens in the phases where Player 1 wins every game of a block. The effect size is the odds ratio $\mathrm{OR} = ad/bc$: $\mathrm{OR} > 1$ means that Player 1 does better when moving first. Since ten phases are tested simultaneously, the $p$-values are adjusted with Holm's sequentially rejective procedure [36], which keeps the probability of at least one false positive across the ten tests below 5%.

**Model 2: test across phases.** The ten tables are then treated as strata of a single analysis with the Cochran–Mantel–Haenszel (CMH) test [35, 34], which tests the null hypothesis that the common odds ratio equals one – that is, that there is no systematic advantage for the first mover once differences in strength between match-ups are accounted for – and provides the Mantel–Haenszel pooled estimate of that common odds ratio. The pooled estimate is meaningful only if the odds ratios are similar across phases; this is checked with the Breslow–Day test of homogeneity (with Tarone's adjustment) [34]. A significant Breslow–Day result indicates that the colour effect is specific to some match-ups – an interaction between colour and the pair of strategies – rather than a generic first-move advantage. Tables with an empty cell receive the standard 0.5 correction in every cell for the pooled estimate.

As a descriptive complement, the analysis also reports Black's win rate in each phase regardless of which strategy plays Black, i.e. Player 1's wins in Block A plus Player 2's wins in Block B, divided by $n$. Both models are implemented in the `analyze_benchmark.py` script, using SciPy and statsmodels. Per-game logs are available in the thesis repository [37].

## 12 Test Results and Analysis

Following the protocol described in Chapter 11, the ten-phase benchmark suite was run to completion in two sessions with the same harness and settings. Phases 1–6, which involve only the handcrafted evaluator, come from the original run of the full suite. Phases 8–11, which involve the CNN evaluator, were re-run once OthelloCNN had been retrained with the protocol of Chapter 10, so that every CNN result reported here refers to the final network described there. The measured play amounts to 439.3 minutes (about 7.3 hours) in total: 184.5 minutes for Phases 1–6 and 254.8 minutes for Phases 8–11. Every phase produced a fully measured result; no figure in this chapter relies on an extrapolated or projected estimate.

### 12.1 Full Results

Table 7 reports the outcome of all ten phases: the match-up, the number of games played, Player 1's win/draw/loss percentages, the 95% Wilson confidence interval on Player 1's win rate, and

the average wall-clock time per game (a direct measure of computational cost). Figure 9 presents the same win rates graphically.

| Phase (P1 vs P2) | N | P1 W% | D% | P2 W% | 95% CI (P1) | s/game |
|---|---|---|---|---|---|---|
| 1. Random vs Random | 400 | 47.5 | 6.0 | 46.5 | [42.7, 52.4] | 0.007 |
| 2. Heuristic vs Random | 400 | 80.8 | 2.8 | 16.5 | [76.6, 84.3] | 0.027 |
| 3. MinimaxAB vs Heuristic | 168 | 99.4 | 0.6 | 0.0 | [96.7, 99.9] | 6.873 |
| 4. MCTS vs Heuristic | 100 | 98.0 | 0.0 | 2.0 | [93.0, 99.4] | 30.190 |
| 5. Descent vs Heuristic | 100 | 95.0 | 1.0 | 4.0 | [88.8, 97.8] | 30.875 |
| 6. Descent vs MinimaxAB | 100 | 78.0 | 0.0 | 22.0 | [68.9, 85.0] | 37.940 |
| 8. HeuristicCNN vs Heuristic | 400 | 82.0 | 1.2 | 16.8 | [77.9, 85.5] | 0.144 |
| 9. MinimaxAB-CNN vs MinimaxAB | 100 | 100.0 | 0.0 | 0.0 | [96.3, 100.0] | 34.224 |
| 10. MCTS-CNN vs MCTS | 100 | 90.0 | 0.0 | 10.0 | [82.6, 94.5] | 56.391 |
| 11. Descent-CNN vs Descent | 100 | 98.0 | 0.0 | 2.0 | [93.0, 99.4] | 61.695 |

Table 7: Complete measured results for all ten benchmark phases (total measured play: 439.3 min). Phases 8–11 use the final OthelloCNN of Chapter 10.

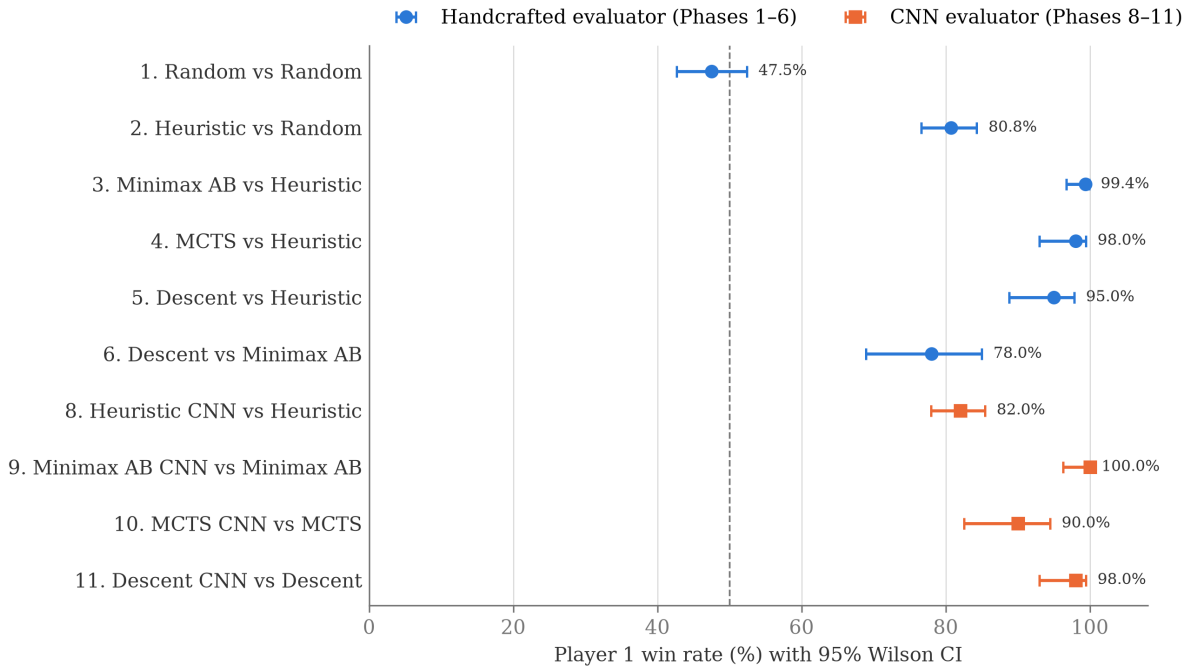



Figure 9: Player 1 win rate with 95% Wilson confidence intervals for all ten measured phases: handcrafted evaluator (circles) and CNN evaluator (squares). The dashed line marks the 50% no-advantage baseline.

Every phase except Phase 1 rejects the null hypothesis of equal strength at the 95% level (its confidence interval excludes 50%), and every one of these nine phases favours Player 1. Phase 1, Random vs. Random, is a sanity check: its confidence interval, $[42.7\%, 52.4\%]$, correctly straddles 50%, confirming that two random players are statistically indistinguishable, as they should be, and that the test harness itself (colour balancing, terminal detection, statistical reporting) is functioning correctly before it is trusted for the substantive comparisons that follow.

## 12.2 Sensitivity to the Treatment of Draws

Table 8 compares, for the five phases in which at least one draw occurred, Player 1's win rate under the convention used throughout this chapter (draws counted as non-wins, $W/n$) with the win rate over decisive games only ($W/(W+L)$), both with 95% Wilson intervals. In the other five phases no draw occurred and the two conventions coincide exactly.

| **Phase (P1 vs P2)** | **Draws** | $W/n$ **[95% CI]** | $W/(W+L)$ **[95% CI]** |
|---|---|---|---|
| 1. Random vs Random | 24 | 47.5 [42.7, 52.4] | 50.5 [45.5, 55.6] |
| 2. Heuristic vs Random | 11 | 80.8 [76.6, 84.3] | 83.0 [79.0, 86.4] |
| 3. MinimaxAB vs Heuristic | 1 | 99.4 [96.7, 99.9] | 100.0 [97.8, 100.0] |
| 5. Descent vs Heuristic | 1 | 95.0 [88.8, 97.8] | 96.0 [90.1, 98.4] |
| 8. HeuristicCNN vs Heuristic | 5 | 82.0 [77.9, 85.5] | 83.0 [79.0, 86.4] |

Table 8: Player 1 win rate (%) with draws counted as non-wins and with draws excluded from the denominator, for the phases containing at least one draw.

As expected, the difference is not statistically relevant. The largest shift, 3.0 percentage points, occurs in Phase 1, the phase with the most draws, and moves the point estimate from 47.5% to 50.5% – if anything closer to the 50% expected from two random players. The two intervals overlap almost entirely in every phase, and no significance verdict changes: Phase 1's interval still contains 50%, and all other intervals still exclude it. The comparison is reported for completeness; the rest of this chapter keeps the draws-as-non-wins convention.

## 12.3 The Algorithm Hierarchy

Phases 2–6, all played under the shared handcrafted `EvaluateGrid` heuristic, establish a clear and statistically decisive strength ordering. The Heuristic agent beats Random 80.8% of the time, confirming that even a simple priority-ladder evaluation is a large improvement over undirected play (Chapter 7). Every deeper search strategy then beats the Heuristic baseline by an even wider margin: Minimax-AB wins 99.4% of the time, MCTS 98.0%, and Descent 95.0% — all three comfortably above 90%, whose confidence intervals do not overlap the Heuristic-vs-Random one. In other words, once a real search procedure is layered on top of the same underlying evaluation function, the marginal gain from adding tree search dwarfs the gain from having a good static evaluation alone.

Phase 6 then resolves the natural follow-up question — if Minimax-AB, MCTS, and Descent all crush the Heuristic by similar margins, how do they compare to each other? — by pitting the two strongest heuristic-evaluated strategies directly against one another: Descent beats Minimax-AB 78.0% of the time (95% CI $[68.9\%, 85.0\%]$). In aggregate, Descent's persistent, incrementally-refined search (Chapter 8) is therefore measurably superior to classical Alpha-Beta search at a comparable time budget, even though Phases 3 and 5 alone might have suggested Minimax-AB was the stronger of the two (99.4% vs. 95.0% against the same Heuristic reference) — a reminder that transitive comparisons through a common opponent do not always predict the result of a direct confrontation. As Section 12.6 shows, however, this advantage is concentrated in the games in which Descent plays White.

## 12.4 The Effect of the CNN Evaluator

Phases 8–11 isolate the contribution of the CNN evaluator by holding the search algorithm fixed and swapping only the leaf (or rollout) evaluation function, exactly the drop-in substitution described in Chapter 10. The CNN-evaluated player wins in every case: 82.0% wrapped in the simple Heuristic strategy, 90.0% wrapped in MCTS (where the CNN replaces the random rollout

rather than a leaf evaluation), 98.0% wrapped in Descent, and 100% – all 100 games – wrapped in Minimax-AB.

Three observations follow from this set of results. First, the uplift from the CNN evaluator is large and one-sided across every search paradigm tested: the lower bound of every confidence interval lies well above 50% (77.9% in the weakest case), so it is not an artefact of one particular algorithm being unusually receptive to neural evaluation. Second, the size of the uplift is not uniform, and it follows the way the network is used. It is largest when the network evaluates the leaves of a deterministic tree search: Minimax-AB-CNN (100%, CI [96.3%, 100%]) and Descent-CNN (98.0%, CI [93.0%, 99.4%]) are statistically indistinguishable from each other and both near the ceiling. It is intermediate when the network replaces the random rollout of MCTS (90.0%, CI [82.6%, 94.5%]), and smallest when it only scores the positions one move ahead in the greedy Heuristic strategy (82.0%, CI [77.9%, 85.5%]), the only case whose interval does not overlap those of the two tree searches. A plausible reading is that a better per-position estimate is worth more the more positions a search compares through it, and the more deterministically it propagates them. Each phase, however, measures the CNN variant against a different heuristic-evaluated opponent, so these win rates are not on a common scale and the ordering should be read as indicative. Third, and consistently with the design argument in Section 10.7, the uplift holds regardless of how differently the surrounding algorithms integrate the network: MCTS uses it to replace an entire stochastic rollout, while Minimax-AB and Descent use it as a deterministic leaf evaluator, yet both integration styles produce a clear, one-sided advantage.

## 12.5 Computational Cost

The per-game timing column of Table 7 (average wall-clock time per game, computed as each phase's total measured duration divided by its number of games) spans roughly four orders of magnitude, from 7 milliseconds (Random vs. Random) to nearly 62 seconds (Descent-CNN vs. Descent) — a direct empirical illustration of the cost hierarchy implied by the algorithms themselves (Chapter 9). The two strategies that do not search (Random, Heuristic) complete a game in tens of milliseconds, and replacing the Heuristic's scoring with a CNN call raises this only to 0.14 seconds (Phase 8); introducing a fixed-depth search against a cheap opponent (Minimax-AB vs. Heuristic, Phase 3) already raises it to nearly 7 seconds per game; and once *both* players in a match-up are searchers (Phases 4–6 and 9–11), per-game cost climbs into the 30–62 second range and dominates the suite's total run time. Because every CNN-evaluated phase pits a CNN-augmented agent against its own heuristic-evaluated counterpart — itself already a costly searcher for Minimax-AB, MCTS, and Descent — Phases 9–11 are among the most expensive in the suite, even though the network's own per-call inference cost (0.2–2 ms, Chapter 10) is negligible next to the search budget it is embedded in: together they account for 253.8 of the 439.3 minutes of measured play, despite covering only three of the ten match-ups. A direct consequence, foreshadowed in Chapter 11, is that the nominal 20-minute figure is not a duration every phase respects: Phases 4–6 and 9–11 are all constrained by the mandatory 100-game floor rather than by the time target itself, and run for 50 to over 100 minutes each (computed directly from Table 7's N and s/game columns) – the 20-minute figure sizes the sample, but does not bound the phase's actual duration once the floor takes over.

## 12.6 Colour Effects Within Phases

Figure 10 and Table 9 report the result of the post-hoc colour analysis described in Section 11.4: for every phase, Player 1's win rate when playing Black (Block A) and when playing White (Block B), the odds ratio, and the Holm-adjusted $p$-value of Fisher's exact test.

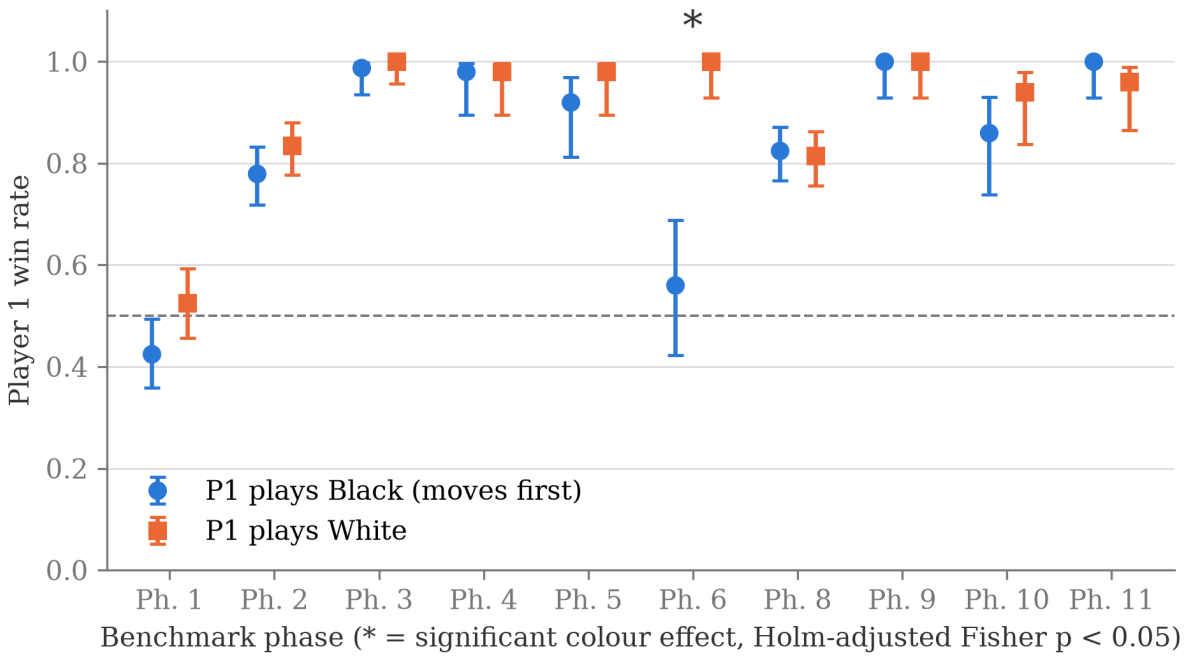



Figure 10: Player 1 win rate with 95% Wilson confidence intervals when playing Black (circles) and White (squares), for each benchmark phase. Asterisks mark the phases with a significant colour effect (Fisher's exact test, Holm-adjusted $p < 0.05$). The dashed line marks the 50% baseline.

| **Phase (P1 vs P2)** | **P1 W% as Black** | **P1 W% as White** | **OR** | $p$ **(Holm)** |
|---|---|---|---|---|
| 1. Random vs Random | 42.5 | 52.5 | 0.67 | 0.51 |
| 2. Heuristic vs Random | 78.0 | 83.5 | 0.70 | 1.00 |
| 3. MinimaxAB vs Heuristic | 98.8 | 100.0 | 0 | 1.00 |
| 4. MCTS vs Heuristic | 98.0 | 98.0 | 1.00 | 1.00 |
| 5. Descent vs Heuristic | 92.0 | 98.0 | 0.23 | 1.00 |
| 6. Descent vs MinimaxAB | 56.0 | 100.0 | 0 | $2.4 \cdot 10^{-7}$ |
| 8. HeuristicCNN vs Heuristic | 82.5 | 81.5 | 1.07 | 1.00 |
| 9. MinimaxAB-CNN vs MinimaxAB | 100.0 | 100.0 | – | 1.00 |
| 10. MCTS-CNN vs MCTS | 86.0 | 94.0 | 0.39 | 1.00 |
| 11. Descent-CNN vs Descent | 100.0 | 96.0 | $\infty$ | 1.00 |

Table 9: Colour analysis, Model 1: Player 1 win rate by colour, odds ratio (OR > 1: Player 1 does better as Black; 0 and $\infty$ arise when a block contains no non-win or no win, and the OR is undefined in Phase 9, where Player 1 wins every game in both blocks) and Holm-adjusted Fisher $p$-value.

**Per-phase results (Model 1).** In nine of the ten phases, Player 1's result does not depend significantly on colour. Phase 1 is a useful reference: with two random players, Black wins 42.2% of the games against White's 51.8% (the remaining 6.0% being draws), a tendency in favour of the second mover that does not reach significance on its own ($p = 0.51$ after adjustment). The single exception is Phase 6, Descent vs. Minimax-AB, significant well beyond any conventional threshold: Descent wins every game as White but only 56.0% as Black (95% CI $[42.3\%, 68.8\%]$), so that whoever plays Black wins only 28.0% of the games. In Phase 9, by contrast, the CNN-evaluated Minimax-AB wins all 50 games in each colour, and no colour effect can arise.

**Pooled results (Model 2).** Across all ten phases, the Mantel–Haenszel pooled odds ratio is 0.65 (95% CI $[0.51, 0.83]$), and the CMH test rejects the absence of a common colour effect ($\chi^2 = 12.00$, $p < 0.001$): taken together, Player 1 wins less often when it moves first. The Breslow–Day test, however, also rejects the homogeneity of the odds ratios ($\chi^2 = 26.95$, $p = 0.001$), and the heterogeneity is entirely due to Phase 6. When Phase 6 is excluded, the remaining nine phases are homogeneous (Breslow–Day $p = 0.58$) and still share a modest, significant colour effect in the same direction (pooled OR $= 0.75$, CMH $p = 0.03$). The benchmark therefore shows two distinct things: a small, consistent advantage for the second mover (White) across the suite, in the same direction as the tendency already visible between two random players, and a much stronger effect specific to Phase 6. Whether the former reflects Othello itself at the level of play of these agents or a property of the specific strategies cannot be established from this benchmark.

**Implications.** Because every phase plays exactly half of its games in each colour, the aggregate win rates of Table 7 remain valid colour-balanced averages, and the modest general advantage of White cancels out of them by construction. For Phase 6, however, the aggregate hides a strong dependence on colour, which qualifies one conclusion of this chapter: Descent's superiority over Minimax-AB is established when Descent plays White, but not when it plays Black, where its confidence interval contains 50%. A plausible explanation lies in the opponent. Minimax-AB searches to a fixed depth and introduces randomness only when breaking ties between equally scored moves, so its play is close to deterministic. Within a colour block, games against it may therefore follow only a small number of distinct trajectories, and the outcome of the whole block may reflect those few lines rather than 50 independent games. The same mechanism is compatible with the perfect 100-0 score of Phase 9, where Minimax-AB plays on both sides. If so, the games of these phases are not independent, their effective sample size is smaller than the nominal $n$, and their confidence intervals are narrower than they should be. The current logs do not record move sequences, so this explanation cannot be verified directly; it is stated as a limitation in Section 15.4.

## 12.7 Summary

Taken together, the measured phases establish a consistent, statistically well-supported picture: under a shared evaluation function, search quality follows the ordering Random $\ll$ Heuristic $\ll$ {Minimax-AB, MCTS, Descent}, with Descent edging out Minimax-AB in direct confrontation; and replacing the handcrafted evaluator with the trained CNN evaluator (Chapter 10) yields a robust win-rate improvement under every search algorithm tested, largest when the network evaluates the leaves of a deterministic tree search (Minimax-AB and Descent, 98–100%), intermediate when it replaces the MCTS rollout (90%), and smallest in the one-ply Heuristic strategy (82%). The post-hoc analyses of Sections 12.2 and 12.6 leave this picture unchanged in aggregate, but show a small general advantage for the second mover and, in the match-up between Descent and Minimax-AB (Phase 6), an advantage for Descent that is significant only when it plays White. These results validate, empirically and with quantified statistical confidence, the central hypothesis motivating the transition to Othello in Chapter 6: that a neural evaluation function trained on real game data can be integrated as a direct, drop-in improvement over a handcrafted heuristic, across search algorithms that use it in structurally different ways.

# Part III
# Conclusions: The Beer Game and the Management Engineering Bridge

## 13 The Beer Game as a Markov Decision Process

### 13.1 Motivation and Scope

Part II answered RQ1 (Section 1.2): a CNN evaluator, trained once and used statically, improves every one of the four search paradigms benchmarked in Chapter 12, with a 95% confidence interval lying entirely above 50% in each case, and a gain that is largest when the network evaluates the leaves of a deterministic tree search (100% for Minimax-AB and 98.0% for Descent, Phases 9 and 11 – a comparison Chapter 15 returns to when it resolves RQ-side). This chapter, together with Chapters 14 and 15, takes that result at face value and turns to RQ2: whether the methodology behind it extends beyond board games to a supply-chain optimization problem.

Answering RQ2 by building a Beer Game simulator, a training corpus to replace WTHOR, and a benchmark of the statistical rigor Chapter 11 achieved would be a project of a size similar to the one just completed. Part III instead extends the *theoretical* half of Part II's methodology to a domain chosen specifically because it violates, in a structured and documented way, the assumptions Section 2.1 found common to Tic-Tac-Toe, Connect 4, and Othello. This chapter describes the Beer Game and its place in the operations-management literature (Section 13.2), formalizes it as a Markov Decision Process in direct structural comparison with the Othello formalization used throughout Part II (Section 13.3), and states plainly which of Section 2.1's axes it relaxes, and which it does not (Section 13.4). Chapter 14 then revisits each of the four algorithms and the CNN evaluator to assess what would need to change for each to apply, and sketches a proposed architecture; Chapter 15 answers RQ1, RQ2, and RQ-side together and closes the thesis.

Chapters 13 and 14 together are best read as a transfer analysis: for each piece of Part II's methodology, which properties survive the move to the Beer Game unchanged, which require adaptation, and which have no analogue at all.

### 13.2 The Beer Game: Problem Description

The Beer Distribution Game was devised by Jay Forrester at MIT in the late 1950s as a physical, board-based teaching exercise for what he called *industrial dynamics* [10], and has since become one of the most widely used simulations in operations and supply-chain management education. Its canonical form models a single product moving through a serial, four-echelon supply chain: a *retailer*, selling to end customers; a *wholesaler*, supplying the retailer; a *distributor*, supplying the wholesaler; and a *factory*, producing the product and supplying the distributor. Time advances in discrete periods. At every period, each echelon receives an order from its downstream neighbor (or, for the retailer, observes end-customer demand), decides how much to order from its own upstream neighbor, ships as much as it can of what it owes, and incurs a cost proportional to the inventory it holds and the demand it fails to satisfy on time. Shipments and orders both take a fixed *lead time* to arrive, so a decision made today does not affect the chain's physical state until several periods later.

What makes the Beer Game canonical is a phenomenon that emerges almost without exception when it is played by human participants: order variance grows systematically upstream, away from the stable end-customer demand that originates the chain. A modest, one-time increase in customer demand is amplified, echelon by echelon, into large swings in the factory's production orders, alternating stockouts with excess inventory, even though no single echelon behaves

irrationally and demand itself changed only once. Sterman characterized this *bullwhip effect* as a robust, reproducible consequence of the local, boundedly rational, feedback-misperceiving heuristics human players apply – including trained MBA students who know the rules and lead times in advance – rather than a result of poor information or malicious intent [11]; Lee, Padmanabhan, and Whang later identified its core mechanisms – demand-signal processing, order batching, price fluctuation, rationing-induced gaming – as generic causes of the same distortion in real supply chains, independent of the toy setting where it was first isolated [12].

This combination – a simple set of local decision rules, an emergent and economically costly system-level failure mode, and direct relevance to the practice of supply-chain management – makes the Beer Game the natural point of contact between this thesis's methodology and the degree programme it is submitted for. Unlike Othello, whose appeal to a management-engineering curriculum is indirect (Section 1.1: board games strip away the complications of operational decision-making while preserving its essential structure), the Beer Game requires no such argument: it is already a standard object of study in supply-chain management, and a decision engine that outperformed the heuristics typically taught alongside it would be of direct relevance to the field.

## 13.3 Formalization as a Markov Decision Process

This section formalizes the Beer Game with the same rigor Section 2.1 applied to sequential decision-making in general. Two variants are distinguished: a *centralized, single-objective* formalization, in which one decision-maker chooses order quantities for all four echelons to minimize total supply-chain cost, and a *decentralized, multi-agent* formalization, in which each echelon observes only its own local state and optimizes only its own local cost. The distinction determines which parts of this thesis's methodology apply at all, and it is discussed in Section 13.4.

### State and action spaces

Index the four echelons $i \in \{1, 2, 3, 4\}$ (retailer, wholesaler, distributor, factory), and let $L_i$ be the shipping lead time between echelon $i$ and its supplier. At the start of period $t$, echelon $i$'s local state is

$$s_{i,t} = \big(I_{i,t},\ B_{i,t},\ P_{i,t}^{(1)}, \ldots, P_{i,t}^{(L_i)}\big), \tag{3}$$

with $I_{i,t} \geq 0$ on-hand inventory, $B_{i,t} \geq 0$ backlog of unmet downstream demand, and $P_{i,t}^{(k)}$ the quantity in the $k$-th stage of the incoming shipment pipeline. The full state of the centralized MDP concatenates all four local states with the most recently observed customer demand,

$$s_t = \big(s_{1,t}, s_{2,t}, s_{3,t}, s_{4,t},\ d_{t-1}\big). \tag{4}$$

In the decentralized formalization, agent $i$ observes only $s_{i,t}$ and its own incoming order, and, in the classical partially observable version of the game, nothing about any other echelon directly.

At every period, echelon $i$ chooses an order quantity $a_{i,t} \in \mathcal{A}_i = \{0, 1, \ldots, a_i^{\max}\}$, placed with its supplier. In the centralized formalization the action is the joint vector $a_t = (a_{1,t}, \ldots, a_{4,t})$, so $|\mathcal{A}(s_t)| = \prod_{i=1}^{4} |\mathcal{A}_i|$ grows multiplicatively in the number of echelons – the first of several places, summarized in Table 10, where the Beer Game's combinatorics differ structurally from Othello's bounded branching factor.

### Transition dynamics

Let $O_{i,t}$ be the order echelon $i$ receives, with $O_{1,t} = d_t$ (realized customer demand, $d_t \sim D$) and $O_{i,t} = a_{i-1,t}$ for $i > 1$. What echelon $i$ can ship is its on-hand inventory plus the arriving shipment, $I_{i,t} + P_{i,t}^{(1)}$; what it must satisfy is the new order plus any backlog, $O_{i,t} + B_{i,t}$. What

is actually shipped is therefore

$$X_{i,t} = \min\big(I_{i,t} + P_{i,t}^{(1)},\ O_{i,t} + B_{i,t}\big), \tag{5}$$

and the state updates as

$$I_{i,t+1} = I_{i,t} + P_{i,t}^{(1)} - X_{i,t}, \tag{6}$$

$$B_{i,t+1} = B_{i,t} + O_{i,t} - X_{i,t}, \tag{7}$$

$$P_{i,t+1}^{(k)} = P_{i,t}^{(k+1)}\ (k < L_i), \qquad P_{i,t+1}^{(L_i)} = a_{i,t}. \tag{8}$$

The only randomness in this system is the demand draw $d_t \sim D$ at the retailer; every other quantity is a deterministic function of the current state and the chosen actions. This matters directly for Chapter 14: the Beer Game's stochasticity enters the transition function at a single, well-identified point, rather than being interleaved throughout play as a card draw or die roll would be, which is what makes an Expectiminimax-style recovery of Minimax at least conceivable.

### Reward function

Each echelon incurs a per-period cost

$$c_{i,t} = h_i\, I_{i,t+1} + p_i\, B_{i,t+1}, \tag{9}$$

with $h_i > 0$ a holding-cost rate and $p_i > 0$ a backlog penalty, generally $p_i > h_i$ since failing to satisfy demand is costlier than holding excess stock; the reward is $r_{i,t} = -c_{i,t}$. In the **centralized** formalization, the system reward is $R_t = \sum_i r_{i,t}$ and a single decision-maker chooses $a_t$ to maximize $\mathbb{E}\big[\sum_{t=0}^{T-1} R_t\big]$ over a fixed horizon $T$ (classically 35–52 periods) – an MDP in the strict sense of Section 2.1, with the same structure as the Othello formalization: one decision-maker, a well-defined state, a well-defined action, a scalar reward. In the **decentralized** formalization – the one actually instantiated when four independent participants, or four independent algorithms, play the game as it is in practice – each echelon maximizes its own $\mathbb{E}\big[\sum_t r_{i,t}\big]$ from local information alone, turning the problem into a Markov (stochastic) game [28, 29], or, if the echelons agree to be evaluated jointly while acting on local information only, a decentralized partially observable MDP (Dec-POMDP). This is the decentralized, locally-optimizing, non-communicating version of the game that produces the bullwhip effect (Section 13.2), while the centralized formalization, useful as a computational benchmark, describes a problem the classical Beer Game was never meant to pose to its players.

### Comparison with the Othello formalization

Table 10 compares this formalization, component by component, with the Othello formalization used throughout Part II (state = the $8 \times 8$ board plus the player to move, action = a legal disc placement, reward = zero everywhere except a terminal disc-margin signal in $[-64, +64]$, Chapter 10).

| Component | Othello | Beer Game (centralized MDP) |
|---|---|---|
| State | $8 \times 8$ board + turn | $(I_i, B_i, P_i^{(1..L_i)})_{i=1}^4 + d_{t-1}$ |
| Action | one legal disc placement | joint order vector $(a_1, a_2, a_3, a_4)$ |
| Branching factor | $\lesssim 12$ (legal moves) | $\prod_i \lvert\mathcal{A}_i\rvert$ (multiplicative) |
| Transition | deterministic (flip rule) | stochastic (demand draw $d_t \sim D$) |
| Reward | 0 until terminal, then $\pm$ margin | $-\sum_i (h_i I_{i,t+1} + p_i B_{i,t+1})$, every period |
| Horizon | natural termination (no legal moves) | fixed $T$, administratively imposed |
| Decision-makers | 2, strictly opposed (zero-sum) | 1 (centralized) or 4 (decentralized, cooperative) |
| Observability | full, for both players | full (centralized) or local (decentralized) |

Table 10: Component-by-component comparison of the Othello MDP formalization used in Part II and the Beer Game MDP formalized above.

Two rows deserve a comment. Othello's reward is *sparse* – one non-zero signal per game, delivered only at termination – which is exactly why every algorithm in this thesis needs a leaf evaluation function in the first place: to estimate a quantity the environment will not reveal for dozens of moves. The Beer Game's reward is *dense*: a cost is realized every period, and the quantity of interest is a cumulative sum over a fixed horizon rather than a single terminal outcome. And where Othello terminates naturally and unambiguously – neither player has a legal move – the Beer Game's horizon $T$ is a modeling convention, imposed for tractability, with no analogue to a board filling up. Both distinctions bear directly on Chapter 14's assessment of Descent, whose defining property – extending every iteration to a genuine terminal state, so every backed-up value is exact rather than an estimate of an estimate (Section 8) – relied specifically on Othello's natural, always-reachable termination.

### 13.4 Structural Differences from the Board-Game Setting

Section 2.1 identified three axes along which a sequential decision-making problem can vary, and every game in Part II sits at the same corner of that space. The Beer Game, in its decentralized formalization, sits close to the opposite corner.

- **Cooperative, not adversarial.** There is no player to minimize against. Every echelon's cost is driven up by the *same* underlying failure – excess inventory or unmet demand – not by another agent's deliberate, opposing choice. Minimax and Alpha-Beta (Sections 2.2–2.3), built around an opponent assumed to play the worst move for the searching player, have no object to act on here.
- **Stochastic, not deterministic.** Every Othello transition is a fully determined function of board and move; Equations (6)–(8) show the Beer Game's transitions are deterministic *given* the demand realization $d_t$, but $d_t$ itself is a genuine random draw. Every algorithm this thesis implements assumes a single, known successor per action, and each must be reconsidered in light of this.
- **Four agents, not two.** Beyond the combinatorial consequence already noted in Table 10, four decision-makers raise a question two strictly opposed players never do: whether to search a single joint action space at all, or decompose the problem into four coupled but separately searched local decisions.
- **A fixed horizon, not natural termination.** The Beer Game's episode length $T$ is an external convention, not a property the environment signals – a direct problem for any search procedure, such as Descent, defined in terms of reaching a genuine terminal state.

- **Potentially partial observability.** In the decentralized formalization that actually produces the bullwhip effect, a retailer does not observe the factory's inventory, nor does any echelon observe another's local cost. Full observability – assumed without comment by every algorithm in this thesis, since both Othello players see the entire board at all times – is a simplifying choice here, valid for the centralized "social planner" benchmark but not for the game as it is actually played [30].

None of these five is a minor adjustment; each removes or weakens an assumption every algorithm in Part II was implemented, tuned, and benchmarked under. Chapter 14 takes them one at a time against each of the four algorithms and the CNN evaluator.

# 14 Algorithm Transfer Analysis and a Proposed Architecture

This chapter resolves RQ2 (Section 1.2) for each piece of Part II's methodology in turn. Section 14.1 revisits, in the order introduced in Chapter 2, the four search algorithms and the CNN evaluation function, and asks what – if anything – survives the five disanalogies of Section 13.4. Section 14.2 sketches, at the level of a design and illustrative pseudocode, how the surviving pieces could be assembled into a single Beer Game decision engine. Section 14.3 states the obstacles such an implementation would have to overcome.

## 14.1 Algorithm-by-Algorithm Transfer Analysis

Table 11 states the verdict for each algorithm; the discussion below justifies each row.

| **Algorithm** | **Transfer verdict** |
|---|---|
| Minimax / Alpha-Beta | Does not transfer directly (no adversary to minimize against); partially recoverable, in the single-agent-vs-nature special case only, via Expectiminimax-style chance nodes for the demand draw. |
| MCTS (UCT) | Transfers with the least modification: stochastic transitions are handled by sampling $d_t$ at each expansion, exactly as classical MCTS already samples a stochastic rollout; the central open question is the joint- versus per-agent action-space design, not stochasticity. |
| Descent | The strongest candidate for transfer in principle: its persistent, incrementally-refined value estimate does not structurally require an adversarial backup, but its deterministic, single-sample edge-value overwrite (Listing 15) requires replacement by an averaged, Bellman-expectation backup before it is sound under stochastic transitions. |
| CNN evaluation function | The design principle (a trained value estimate as a drop-in replacement for a handcrafted cost heuristic) transfers; the specific 2-D convolutional architecture does not, since the Beer Game state has no spatial or dihedral structure for convolutions to exploit. |

Table 11: Transfer verdict for each algorithm developed in this thesis, assessed against the Beer Game MDP of Section 13.3 and the disanalogies of Section 13.4.

### Minimax and Alpha-Beta

Both algorithms are defined by an operation – minimize the opponent's outcome – that presupposes an opponent whose interests are opposed to the searching player's. Section 13.4 already ruled that opponent out, in both formalizations: the environment's uncertainty (the demand draw $d_t$) is not a hostile agent selecting the worst outcome, and the other three echelons, in the decentralized case, are not opposed to the searching echelon's objective either. A partial recovery is available in a narrower setting: if a single echelon is optimized in isolation, treating the rest of the chain and customer demand as an exogenous stochastic process, the resulting single-agent-versus-nature problem admits the classical *Expectiminimax* extension already noted in footnote 1 of Table 1 – chance nodes, at which the value is the expectation over $d_t$'s distribution rather than a max or a min, interleaved with ordinary maximizing nodes at the decision-maker's own turns. This recovers a well-defined search procedure for the single-agent special case only; it does nothing for the joint, cooperative, four-agent structure of the problem as actually posed, and the size of the expectation at each chance node reintroduces exactly the combinatorial cost alpha-beta pruning was designed, under a strict adversarial bound, to avoid – pruning under expectation is provably far weaker, since a chance node's value is not bounded above by any single child's value the way a min node's is.

### Monte Carlo Tree Search

Section 2.4 already identified MCTS as the algorithm in this thesis best positioned to bridge toward stochastic environments, since its core operation – sample a trajectory, observe an outcome, average it into a running estimate – has no structural dependence on determinism. UCT and its variants are an established tool for planning in stochastic MDPs and POMDPs, where a node represents a state of the underlying process and the expansion step samples one realization of the environment's randomness (here, a customer-demand draw) rather than following a single deterministic successor. Adapting the Othello implementation of Chapter 7 to the Beer Game would replace its deterministic board update with a call that samples $d_t \sim D$ before applying Equations (6)–(8), and would replace the two-outcome ($\pm 1$, draw 0) rollout result with the negated, normalized cumulative cost accumulated along a random or heuristic continuation to the horizon $T$. The genuinely open design question is not stochasticity but multiplicity of agents: whether the tree searches over the full joint action space $\mathcal{A}(s) = \prod_i \mathcal{A}_i$ (preserving optimality with respect to the centralized objective, at the cost of a branching factor multiplied across four echelons) or decomposes into four coupled per-echelon trees searched with some coordination mechanism (controlling the branching factor, at the cost of any guarantee that the resulting joint policy is even locally optimal). Neither choice has a precedent in this thesis's two-player algorithms, where the number of decision-makers was always exactly two.

This transfer verdict, however, is cleanest for the centralized, fully observable formalization of Section 13.3, where a single decision-maker with full knowledge of $s_t$ chooses a joint action $a_t$ – structurally the closest analogue to Othello's two-player, fully observable setting, differing only in that the opponent is replaced by a chance node. The decentralized formalization that actually produces the bullwhip effect (Section 13.2) is not just multi-agent but also partially observable: each echelon's local state $s_{i,t}$ (Equation (3)) is not, in the classical version of the game, visible to the other three. MCTS's core sampling operation is agnostic to this distinction, but the search itself is not: a node in the decentralized case represents a belief over the other echelons' hidden state rather than a fully known state – the setting studied by partially observable extensions of MCTS rather than by UCT as described in Section 2.4 [30, 31]. Adapting the Othello implementation of Chapter 7 to the centralized case is therefore closer to a direct transfer than adapting it to the decentralized case, which requires the further, unaddressed step of searching over beliefs rather than states.

### Descent Minimax

Descent's defining mechanism was described in Section 8 in terms that never mention the word "adversarial": repeatedly identify the currently most promising line, extend it by one step, back up a refined value estimate toward the root. Nothing in that description requires the backup step to be a max / min alternation, which is why Descent is the strongest candidate for transfer of the four. The same operation has a direct analogue in asynchronous and real-time dynamic-programming methods for general MDPs – most concretely Barto, Bradtke, and Singh's Real-Time Dynamic Programming (RTDP), which likewise refines a value function by repeatedly updating the states along a single, currently-promising trajectory rather than sweeping the full state space [14].

This transfer question is distinct from RQ-side (Section 1.2). RQ-side asks whether Descent's terminal-search property already pays off in Othello without reinforcement learning – Chapter 15 answers that directly from Part II's existing benchmark, with no new Beer Game evidence required. The question here is narrower and structural: can Descent's *mechanism* be adapted to the Beer Game's stochastic, cooperative, non-terminating setting at all? Two changes are needed before it could fit.

First, the backup operator. Othello's `descent_iter` updates an edge's value with a single overwrite, correct precisely because Othello's transition is deterministic: revisiting the same edge always leads to the same child, so the most recently refined value *is* the edge's exact value. Under a stochastic transition, the same edge can lead to different children on different visits, depending on the demand realization sampled; an overwrite would discard every earlier sample and track only the most recent draw rather than its expectation. The correct replacement is an incrementally updated running average – the same $W/N$ bookkeeping MCTS already performs (Section 2.4) – turning the edge update into a genuine sample-based Bellman-expectation backup, $\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \frac{1}{N(s,a)}\big[r + \hat{V}(s') - \hat{Q}(s,a)\big]$.

Second, the notion of a terminal state. Descent's efficiency argument in Section 8 – every value it computes is exact because every iteration reaches a true terminal state – depended on Othello's terminal states being cheaply reachable, typically within a few dozen further plies. The Beer Game's horizon $T$ is an order of magnitude or more further away in periods than Othello's remaining plies typically are late in the game, and, more importantly, is not an emergent property of the state (as "no legal moves remain" is in Othello) but an external convention: an iteration insisting on reaching period $T$ before backing up any value would, early in an episode, spend most of its budget on one very deep rollout rather than refining the choice actually at hand. A Beer-Game-adapted Descent would therefore bootstrap from the trained value network of Section 14.2 once a shorter, fixed sampling depth is reached, rather than insisting on $t = T$ every iteration – a departure from the letter of Descent's terminal-anchoring property, though not from its spirit, since the value network's own training target is itself derived from longer or complete rollouts.

### The CNN evaluation function

Chapter 10 established a reusable pattern: replace a handcrafted, linearly-weighted evaluation function with a network trained by regression on real outcome data, then substitute it into every search algorithm through a single dispatcher, without touching any surrounding search logic. Chapter 12 showed that pattern to be robust across four structurally different ways of consuming an evaluation function – this is RQ1's answer. The *design principle* – a learned $\hat{V}(s)$ substituting for a handcrafted cost estimate, trained by regression on outcome data rather than tuned by inspection – transfers to the Beer Game without qualification, since it never depended on adversarial or deterministic structure in the first place, only on there being a scalar quantity worth estimating and data to estimate it from. What does *not* transfer is OthelloCNN's specific architecture: its residual, no-pooling, $3 \times 3$-convolutional design was justified entirely by properties specific to an $8 \times 8$ board – fixed spatial size, dihedral symmetry exploitable for eightfold data augmentation, and a strategic value that is a genuinely local, convolutional

function of a cell's neighborhood. The Beer Game's state (Section 13.3) is a low-dimensional vector of inventories, backlogs, and pipeline contents with no spatial layout and no symmetry group for a convolution to exploit; a value network for it is more naturally a fully-connected multilayer perceptron, as Section 14.2 proposes. This is the one component whose *role* in the methodology transfers cleanly while its specific implementation does not.

## 14.2 A Proposed Architecture

This section sketches, at the level of a design and illustrative pseudocode, how the verdicts of Section 14.1 could be assembled into a single Beer Game decision engine, mirroring the Othello engine of Part II wherever the transfer analysis found grounds to do so, and departing from it wherever it did not.

### State encoder

A `StateEncoder` component plays the role `_board_to_tensor_local` plays in the Othello engine: it maps the raw simulation state $s_t$ (Equation (4)) to a fixed-length, normalized input vector $x_t \in \mathbb{R}^d$, concatenating each echelon's inventory, backlog, and pipeline contents (each divided by a fixed normalization constant, the analogue of OthelloCNN's $\{0,1\}$-valued disc channels) together with the lagged demand and, optionally, the current period index $t$ if the value network is to be horizon-aware.

### Value network: `BeerGameNet`

Where OthelloCNN is a convolutional network predicting a terminal disc margin, its structural counterpart here, `BeerGameNet`, is a small fully-connected network – two or three hidden layers with ReLU activations, mirroring the two-linear-layer head already used at the end of OthelloCNN – predicting a single scalar, $\hat{V}_\theta(s_t) \approx -\mathbb{E}\big[\sum_{\tau \geq t} R_\tau\big]$: the negated expected future cost-to-go, kept negated so the same maximizing convention used throughout this thesis (higher $\hat{V}$ is better) applies unchanged. Trained by regression, exactly as OthelloCNN is, its loss is again mean squared error against a target return; where that target comes from is a separate question, taken up in Section 14.3.

### Search engine

The search engine adapts MCTS and Descent – not Minimax or Alpha-Beta, per Table 11 – to the Beer Game MDP. Each search node represents a state $s_t$ rather than a board position; expansion samples a demand realization $d_t \sim D$ before applying the transition equations of Section 13.3; and every leaf evaluation calls `BeerGameNet`, the direct structural analogue of `EvaluateGridCNN`'s role as a drop-in replacement for `EvaluateGrid`. Listing 15 sketches the Descent variant, with the two changes Section 14.1 identified as necessary: an averaged, rather than overwritten, edge-value backup, and a value-network bootstrap in place of insisting on a true terminal state at $t = T$ on every iteration.

```
def descent_iter_beergame(s, t, S, value_net, max_depth, t0):
    key = state_key(s, t)

    if t == HORIZON_T or (t - t0) >= max_depth:
        # Bootstrap from the value network rather than
        # forcing every iteration to a true terminal state
        v = value_net(encode(s)) if t < HORIZON_T else 0.0
        S[key] = {"value": v, "N": 1}
        return v

    if key not in S:
```

```
        S[key] = {"edge": {}, "N_edge": {}}
        for a in joint_actions(s):            # or per-agent, see Sec. 14.1
            s_next = sample_transition(s, a)  # nature samples d_t
            r0 = reward(s, a, s_next)         # dense reward: must be included
    here too
            v0 = r0 + value_net(encode(s_next))
            S[key]["edge"][a]   = v0
            S[key]["N_edge"][a] = 1

    edges = S[key]["edge"]
    best_a = max(edges, key=lambda a: edges[a])   # cooperative: always max

    s_next = sample_transition(s, best_a)          # RE-sample: stochastic!
    r = reward(s, best_a, s_next)
    v_next = descent_iter_beergame(s_next, t + 1, S, value_net, max_depth, t0)

    # Bellman-expectation backup: incremental average, NOT overwrite
    n = S[key]["N_edge"][best_a] + 1
    old_q = edges[best_a]
    edges[best_a] = old_q + (r + v_next - old_q) / n
    S[key]["N_edge"][best_a] = n

    return max(edges.values())
```

Listing 15: Illustrative pseudocode for a stochastic, value-network-guided Descent search over the Beer Game MDP. Contrast with `descent_iter`: the edge update is an incremental average rather than an overwrite, and a bootstrap replaces insistence on reaching `t == T`.

Reading off the order quantities at the root – the joint action with the highest refined edge value – mirrors how `PlayAIDescent` reads off the best move from its edge table in the Othello engine. Adapting the same engine to MCTS instead of Descent requires no change to `BeerGameNet` or the state encoder, only to the selection and backup rules, which already average rather than overwrite (Section 2.4) – the same pluggability Section 2.6 and Chapter 10 built into the Othello engine through `_get_eval_fn`, preserved here by design.

## 14.3 Expected Challenges and Open Questions

Three obstacles would have to be resolved before the architecture of Section 14.2 could be trained and evaluated with the rigor Chapter 11 applied to Othello.

**No WTHOR-equivalent training corpus.** OthelloCNN's regression target – the final disc margin of a real, recorded, competitively played game – is available in the millions because Othello has a large community of tournament players and a federation that archives their games. No comparable archive of "expertly played" Beer Game episodes exists: the game's own research literature documents human play precisely *because* it is reliably suboptimal [11], so recorded human sessions would supply examples of the bullwhip effect to avoid, not a ground-truth signal of good play to imitate. Clark and Scarf showed that, for restrictive but tractable cost and demand assumptions (linear costs, i.i.d. demand), a base-stock (order-up-to) policy is provably optimal for a serial multi-echelon inventory system [13], giving a narrow but computable target to regress toward. Alternatively, since Descent produces exact, terminal-anchored values as a byproduct of search rather than requiring a separate data-generation phase (Section 8), `BeerGameNet` could in principle be trained on trajectories the search engine itself generates, bootstrapped from an initially weak network – an AlphaZero-style self-play loop this thesis's own Descent implementation deliberately avoided in Chapter 10, which trained OthelloCNN once, offline, from real data, and never from its own search output. Adopting self-play here would be a genuine departure from this thesis's methodology, and risks the network learning to agree with its own search's blind spots rather than with any external notion of good play.

**Shared versus per-agent value.** Section 13.3 formalized the reward in two variants – a

summed, system-level $R_t$ for a centralized planner, and a per-echelon $r_{i,t}$ for the decentralized game as actually played – and the same fork already surfaced for MCTS's search tree. It recurs for the value network: a single, shared `BeerGameNet` trained on the system-level return is well-defined and directly analogous to OthelloCNN, but answers a different question – what is optimal for the whole chain – than a faithful, decentralized reconstruction asks of each of its four participants. Four separate per-agent networks $\hat{V}_{\theta_i}$ are closer to the decentralized incentive structure, but raise a question with no counterpart in this thesis's zero-sum algorithms: what a "good" joint outcome even means once four agents can each improve their own local value at another's expense – equilibrium-concept territory (Nash equilibria of a Markov game) that a single-objective search-and-evaluate methodology does not by itself supply.

**No clean ground truth for move quality.** Othello's final disc margin is a decisive, immediately available signal of how good a sequence of moves was, which is what makes it possible both to train OthelloCNN by simple regression and to validate a search algorithm's strength directly, by playing it against others and measuring win rate with a Wilson confidence interval (Chapter 11). The Beer Game offers no equivalent: cumulative cost over $T$ is well-defined, but there is no known closed-form optimum outside the narrow Clark–Scarf setting [13], and no comparably rich population of Beer Game agents of independently established strength to benchmark a new decision engine against. Any evaluation of the proposed architecture would therefore have to rely, at least initially, on comparison against the same base-stock heuristics that stand in for expert play above – a considerably weaker form of validation than the statistically quantified tournament Chapter 11 ran for Othello.

# 15 General Conclusions

This chapter closes the thesis by answering, in turn, the three questions Section 1.2 posed at the outset, each with the evidence gathered for it in Part II or Part III, and then states the throughline that connects all three.

## 15.1 RQ1: Yes – a Static Evaluator Helps Regardless of Search Paradigm

RQ1 asked whether a static, pretrained CNN evaluator improves a search algorithm's performance regardless of that algorithm's own exploration mechanism. The benchmark of Chapter 12 answers this directly: replacing the handcrafted evaluator with OthelloCNN produced a one-sided win-rate improvement, with every 95% Wilson confidence interval lying well above 50%, in each of the four search paradigms tested – 82.0% wrapped in the simple heuristic strategy, 90.0% in MCTS, where the network replaces an entire stochastic rollout rather than a leaf value, 98.0% in Descent, and 100% in Minimax-AB. The uplift held whether the network was consumed as a deterministic leaf evaluator (Heuristic, Minimax-AB, Descent) or as a full substitute for a stochastic sampling step (MCTS): evidence that the benefit is a property of the evaluation function itself. Its size, however, depends on how the network is used: it is largest when the network evaluates the leaves of a deterministic tree search, and smallest when it only guides a one-ply greedy choice.

## 15.2 RQ2: Partial – Some of the Methodology Transfers, Some Does Not

RQ2 asked whether this methodology extends to the Beer Game. Chapter 14 answered algorithm by algorithm: Minimax and Alpha-Beta do not transfer, since both are defined by an adversarial operation the Beer Game's cooperative structure does not have; MCTS transfers with the least friction, since its sampling-based core was never tied to determinism or adversariality; Descent transfers in principle but not exactly as it is, once its backup rule is generalized from a deterministic overwrite to an averaged Bellman expectation and its terminal-anchoring is relaxed into a value-network bootstrap; and the CNN evaluator's design principle – a trained value estimate as

a drop-in replacement for a handcrafted one – transfers cleanly, while its specific convolutional architecture, justified entirely by Othello's board structure, does not.

This verdict was not tested empirically. RQ2's answer is therefore methodological, not empirical: some of this thesis's tools generalize, identifiably and for stated reasons, and some do not; whether the ones that do generalize actually outperform the boundedly rational heuristics the Beer Game is used to teach against remains open, and answerable, future work.

## 15.3 RQ-side: A Qualified Yes – Descent Outperforms Fixed-Depth Search, but Does Not Benefit More from the CNN

RQ-side asked whether Descent's defining property – extending every search iteration to a genuine terminal state rather than a fixed depth – is advantageous on its own, even when the evaluation function it feeds is static (Section 2.6) rather than the dynamic, self-improving network Athénan trains by reinforcement learning.

The benchmark supports the first half of this claim, with a qualification. Paired with the same handcrafted heuristic used by every other strategy, Descent beats Minimax-AB head-to-head 78.0% of the time (95% CI [68.9%, 85.0%], Phase 6) – a persistent, incrementally-refined search to a real terminal state outperforming a fixed-depth exact search, with no learning of any kind on either side. The colour analysis of Section 12.6 shows, however, that this advantage is concentrated in the games Descent plays as White: as Black, its win rate of 56.0% has a confidence interval that contains 50%.

The benchmark does not support a second, stronger claim, namely that Descent compounds the benefit of a better evaluator more than the other paradigms do. Paired with the same static CNN, Descent beats its heuristic-evaluated self 98.0% of the time (Phase 11), but Minimax-AB does so in all 100 games (Phase 9), and the two confidence intervals overlap. The two deterministic tree searches benefit from the CNN equally, and more than MCTS or the one-ply Heuristic strategy do; nothing in this comparison singles out Descent. RQ-side is therefore answered with a qualified yes: under a static evaluator, Descent (persistent, best-first, and extending every iteration to a genuine terminal state) outperforms a fixed-depth exact search in aggregate, without any reinforcement learning on either side, but it does not gain more from a learned evaluator than a fixed-depth search does. This is consistent with the terminal-search property contributing to Descent's advantage, since reaching a true terminal state on every iteration is precisely what Athénan relies on to generate exact training targets without reinforcement learning (Section 8); it does not, however, isolate that property from Descent's other differences from Minimax-AB and MCTS – best-first versus full-width expansion, a persistent versus a rebuilt search tree, a time budget versus a fixed depth or iteration count, and an overwrite versus a backpropagated value update – all of which vary simultaneously in the comparisons of Chapter 12. Attributing the advantage to the terminal-search property specifically would require an ablation that holds these other properties fixed (e.g. a depth-capped variant of Descent) and is left to future work (Section 15.4).

## 15.4 Limitations

The results and arguments of the preceding chapters are, on the whole, robust to the concerns raised above, but several methodological choices narrow what they can be taken to show. They are stated in the following paragraphs.

- **No ablation isolates Descent's terminal-search property.** Section 15.3 argues that Descent's advantage over Minimax-AB is consistent with its terminal-search property, but the comparisons in Chapter 12 vary several of Descent's properties simultaneously relative to the other strategies – best-first versus full-width expansion, a persistent versus a rebuilt search tree, a time budget versus a fixed depth or iteration count, and an overwrite versus a backpropagated value update. Attributing the advantage to the terminal-search property

specifically, rather than to Descent as a whole, would require an ablation (e.g. a depth-capped variant of Descent) that this thesis does not report.

- **The nominal per-phase time budget is not always met.** Chapter 11 sizes each phase's sample against a 20-minute target runtime, but subject to a mandatory floor of 100 games. For every phase involving MCTS or Descent, this floor takes precedence over the target, and the phase's actual duration is 50 to over 100 minutes (Table 7). The 20-minute figure should therefore be read as a sizing target rather than a runtime guarantee honoured uniformly across phases.
- **Colour effects, and possibly repeated games, in match-ups involving Minimax-AB.** The post-hoc colour analysis (Section 12.6) finds a small, consistent advantage for the second mover across the suite, which the balanced two-block design cancels out of every aggregate win rate, and a much stronger colour effect in Phase 6, where Descent's advantage over Minimax-AB is significant only when Descent plays White. Since Minimax-AB is close to deterministic, the games of the phases in which it plays (Phases 6 and 9) may repeat a small number of distinct trajectories, which would make them non-independent and their Wilson intervals too narrow. The harness does not record move sequences, so the number of distinct games cannot be measured from the existing logs; logging them, and starting each pair of games from a randomised opening, would remove this limitation.
- **A single primary domain for RQ1.** RQ1's evidence (Section 15.1) comes entirely from Othello. The CNN evaluator's benefit is demonstrated across four search paradigms within that one domain, not across multiple game domains, so generalization to games with different structural properties (larger board sizes, higher branching factors, imperfect information) remains untested.
- **A single CNN architecture.** OthelloCNN (Chapter 10) is one residual, no-pooling design with about 300,000 parameters. No alternative architecture – deeper, wider, with attention, or without residual connections – was trained or compared against it, so the reported uplift is attributable to *a* learned evaluator of this design, not necessarily to the strongest evaluator this data could support.
- **No empirical validation of the Beer Game architecture.** As already stated in Section 15.2, Part III's proposed architecture (Chapter 14) is a design, not an implementation: no Beer Game simulator was built, no training corpus assembled, and no benchmark run. RQ2's methodological verdict – which of this thesis's tools transfer and which do not – is therefore not itself empirically tested, and remains an identified direction for future work rather than a result.
- **No treatment of partial observability in the Beer Game transfer.** Section 14.1 argues that MCTS transfers to the Beer Game with comparatively little friction, but that claim is cleanest for the centralized, fully observable formalization of Section 13.3. The decentralized formalization that actually produces the bullwhip effect (Section 13.2) is also partially observable, and no belief-state or POMDP-aware extension of the proposed search engine is worked out; this is left, alongside the multi-agent action-space question already identified in Section 14.1, as open work.

## 15.5 Closing Remark

The idea this thesis pursued from its first chapter – combine a search procedure with a value estimate, and let that estimate improve as more information becomes available – needed no value estimate at all for Tic-Tac-Toe's exhaustive Minimax, became unavoidable once Connect 4's tree grew too large to solve exactly, and was tested, four ways, on Othello with statistical rigor. Part III carried the same idea to a fourth environment with no board and no opponent, and found that two of the four tools built to implement it are not the right tools for a cooperative, stochastic supply chain, that a third needs a specific and identifiable repair, and that the fourth – the static evaluator whose benefit RQ1 established across every search paradigm tested – is the piece of

this methodology most likely to survive the move. That is the appropriate note on which to close a thesis whose goal was never to claim that board-game algorithms solve supply-chain problems, but to determine, with as much precision as the developed methodology allows, exactly which parts of them do.

# References


[1] Quentin Cohen-Solal and Tristan Cazenave. *Minimax Strikes Back*. LAMSADE, Université Paris-Dauphine, PSL, CNRS, Paris, France. AAMAS 2023 (adapted / related workshop presentation).

[2] Quentin Cohen-Solal. *Learning to Play Two-Player Perfect-Information Games without Knowledge*. CRIL, Univ. Artois and CNRS; Normandie University, UNICAEN, CNRS, GREYC; LAMSADE, Université Paris-Dauphine, PSL, CNRS. arXiv:2008.01188v5 [cs.AI], 2025.

[3] Richard E. Korf and David Maxwell Chickering. *Best-First Minimax Search*. Artificial Intelligence, 84(1–2):299–337, 1996.

[4] Michael L. Littman. *Algorithms for Sequential Decision Making*. Ph.D. thesis, Brown University, 1996.

[5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition, MIT Press, 2018.

[6] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[7] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Fourth edition, Pearson, 2021.

[8] Levente Kocsis and Csaba Szepesvári. *Bandit Based Monte-Carlo Planning*. In: European Conference on Machine Learning (ECML 2006), Springer, pp. 282–293.

[9] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. *A Survey of Monte Carlo Tree Search Methods*. IEEE Transactions on Computational Intelligence and AI in Games, 4(1):1–43, 2012.

[10] Jay W. Forrester. *Industrial Dynamics*. MIT Press, Cambridge, MA, 1961.

[11] John D. Sterman. *Modeling Managerial Behavior: Misperceptions of Feedback in a Dynamic Decision Making Experiment*. Management Science, 35(3):321–339, 1989.

[12] Hau L. Lee, V. Padmanabhan, and Seungjin Whang. *Information Distortion in a Supply Chain: The Bullwhip Effect*. Management Science, 43(4):546–558, 1997.

[13] Andrew J. Clark and Herbert Scarf. *Optimal Policies for a Multi-Echelon Inventory Problem*. Management Science, 6(4):475–490, 1960.

[14] Andrew G. Barto, Steven J. Bradtke, and Satinder P. Singh. *Learning to Act Using Real-Time Dynamic Programming*. Artificial Intelligence, 72(1–2):81–138, 1995.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. *Deep Residual Learning for Image Recognition*. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016), pp. 770–778.

[16] Sergey Ioffe and Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. In: Proceedings of the 32nd International Conference on Machine Learning (ICML 2015), PMLR 37:448–456.

[17] Karen Simonyan and Andrew Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. International Conference on Learning Representations (ICLR 2015).

[18] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR 2015).

[19] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018), pp. 4510–4520.

[20] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. *ImageNet: A Large-Scale Hierarchical Image Database*. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2009), pp. 248–255.

[21] François Chollet. *Deep Learning with Python*. Second edition, Manning Publications, 2021.

[22] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. *Finite-time Analysis of the Multiarmed Bandit Problem*. Machine Learning, 47(2–3):235–256, 2002.

[23] Rémi Coulom. *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. In: Computers and Games (CG 2006), Lecture Notes in Computer Science, vol. 4630, Springer, pp. 72–83.

[24] Richard Bellman. *Dynamic Programming and Stochastic Control Processes*. Information and Control, 1(3):228–239, 1958.

[25] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. *A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play*. Science, 362(6419):1140–1144, 2018.

[26] Michael Buro. *The Evolution of Strong Othello Programs*. In: Ryohei Nakatsu and Junichi Hoshino (eds.), Entertainment Computing, IFIP — The International Federation for Information Processing, vol. 112, Springer, Boston, MA, 2003, pp. 81–88.

[27] Richard E. Korf. *Multi-Player Alpha-Beta Pruning*. Artificial Intelligence, 48(1):99–111, 1991.

[28] Lloyd S. Shapley. *Stochastic Games*. Proceedings of the National Academy of Sciences, 39(10):1095–1100, 1953.

[29] Michael L. Littman. *Markov Games as a Framework for Multi-Agent Reinforcement Learning*. In: Proceedings of the 11th International Conference on Machine Learning (ICML 1994), pp. 157–163.

[30] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. *Planning and Acting in Partially Observable Stochastic Domains*. Artificial Intelligence, 101(1–2):99–134, 1998.

[31] David Silver and Joel Veness. *Monte-Carlo Planning in Large POMDPs*. In: Advances in Neural Information Processing Systems 23 (NeurIPS 2010), pp. 2164–2172.

[32] Edwin B. Wilson. *Probable Inference, the Law of Succession, and Statistical Inference*. Journal of the American Statistical Association, 22(158):209–212, 1927.

[33] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. *Mastering the Game of Go with Deep Neural Networks and Tree Search*. Nature, 529(7587):484–489, 2016.

[34] Alan Agresti. *Categorical Data Analysis*. Third edition, John Wiley & Sons, 2013.

[35] Nathan Mantel and William Haenszel. *Statistical Aspects of the Analysis of Data from Retrospective Studies of Disease*. Journal of the National Cancer Institute, 22(4):719–748, 1959.

[36] Sture Holm. *A Simple Sequentially Rejective Multiple Test Procedure*. Scandinavian Journal of Statistics, 6(2):65–70, 1979.

[37] Giacomo Sacchet. *Beyond Minimax – thesis code and results*. GitHub repository, https://github.com/giacomo-sacchet/game-tree-search-cnn, tag `v1.0-thesis`, 2026.